JoyPythonNptel

November 1, 2024

[1]:
```
t1 = "Amit"
t2 = "Simran"
t3 = "Vidhya"
i = "Sudarshan"
#list
shopping = ["Bread","Coffee","Sugar"]
print(shopping)
```

```
['Bread', 'Coffee', 'Sugar']
```

[2]:
```
for x in shopping:
    print(x)
```

```
Bread
Coffee
Sugar
```

[3]:
```
for i in range(3):
    print(i)
```

```
0
1
2
```

[4]:
```
#append
shopping.append("Curd")
print(shopping)
```

```
['Bread', 'Coffee', 'Sugar', 'Curd']
```

[5]:
```
shopping[1]
```

[5]: 'Coffee'

[6]:
```
shopping[0]
```

[6]: 'Bread'

[7]:
```
for i in range(4):
    print(shopping[i])
```

```
Bread
Coffee
Sugar
Curd
```

[8]:
```
for item in shopping:
    print(item)
```

```
Bread
Coffee
Sugar
Curd
```

[9]:
```
#name
#s1
#s2
shopping.append("Shampoo")
for item in shopping:
    print(item)
```

```
Bread
Coffee
Sugar
Curd
Shampoo
```

[10]:
```
#insert
shopping.insert(1,"Oil")
for item in shopping:
    print(item)
```

```
Bread
Oil
Coffee
Sugar
Curd
Shampoo
```

[11]:
```
#count
ages = [12,23,34,42,15,87,12,16,25,23,81,57,7,3,2,3,1,20]
ages.count(25)
```

[11]: 1

[13]:
```
ages.count(3)
```

[13]: 2

[14]:
```
#len
len(ages)
```

[14]: 18

[15]:
```
len(shopping)
```

[15]: 6

[16]:
```
for i in range(4):
    print(shopping[i])
```

```
Bread
Oil
Coffee
Sugar
```

[17]:
```
print(shopping)
```

```
['Bread', 'Oil', 'Coffee', 'Sugar', 'Curd', 'Shampoo']
```

[19]:
```
for i in range(len(shopping)):
    print(shopping[i])
```

```
Bread
Oil
Coffee
Sugar
Curd
Shampoo
```

[21]:
```
for item in shopping:
    print(item)
```

```
Bread
Oil
Coffee
Sugar
Curd
Shampoo
```

[22]:
```
print(ages)
```

```
[12, 23, 34, 42, 15, 87, 12, 16, 25, 23, 81, 57, 7, 3, 2, 3, 1, 20]
```

[23]:
```
#sort
ages.sort()
print(ages)
```

```
[1, 2, 3, 3, 7, 12, 12, 15, 16, 20, 23, 23, 25, 34, 42, 57, 81, 87]
```

[24]:
```
#reverse
ages.reverse()
print(ages)
```

```
[87, 81, 57, 42, 34, 25, 23, 23, 20, 16, 15, 12, 12, 7, 3, 3, 2, 1]
```

[25]:
```
students = ["Arun","Rajesh","Harish","Akansha","Lakshmi","Varsha"]
print(students)
```

```
['Arun', 'Rajesh', 'Harish', 'Akansha', 'Lakshmi', 'Varsha']
```

[26]:
```
students.insert(0,"JOC")
print(students)
```

```
['JOC', 'Arun', 'Rajesh', 'Harish', 'Akansha', 'Lakshmi', 'Varsha']
```

[27]:
```
#slicing
#list_name[start_index:end_index+1]
students[1:5]
```

[27]:
```
['Arun', 'Rajesh', 'Harish', 'Akansha']
```

[28]:
```
students[:]
```

[28]:
```
['JOC', 'Arun', 'Rajesh', 'Harish', 'Akansha', 'Lakshmi', 'Varsha']
```

[29]:
```
students
```

[29]:
```
['JOC', 'Arun', 'Rajesh', 'Harish', 'Akansha', 'Lakshmi', 'Varsha']
```

[30]:
```
students[3:]
```

[30]:
```
['Harish', 'Akansha', 'Lakshmi', 'Varsha']
```

[31]:
```
students[:5]
```

[31]:
```
['JOC', 'Arun', 'Rajesh', 'Harish', 'Akansha']
```

[32]:
```
#default start index is 0
#default end index = len(list)
students[2:5]
```

[32]: ['Rajesh', 'Harish', 'Akansha']

0.1 FIZZBUZZ

[35]:
```
for i in range(1,51):
    print(i)
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40

```
41
42
43
44
45
46
47
48
49
50
```

[38]:
```python
for i in range(1,51):
    if(i % 3 == 0):
        print(str(i)+" Fizz")
    else:
        if(i % 5 == 0):
            print(str(i)+" Buzz")
        else:
            if(i % 3 == 0 and i % 5 ==0):
                print(str(i)+" Fizz Buzz")
            else:
                print(str(i))
```

```
1
2
3 Fizz
4
5 Buzz
6 Fizz
7
8
9 Fizz
10 Buzz
11
12 Fizz
13
14
15 Fizz
16
17
18 Fizz
19
20 Buzz
21 Fizz
22
23
24 Fizz
25 Buzz
```

```
26
27 Fizz
28
29
30 Fizz
31
32
33 Fizz
34
35 Buzz
36 Fizz
37
38
39 Fizz
40 Buzz
41
42 Fizz
43
44
45 Fizz
46
47
48 Fizz
49
50 Buzz
```

[44]:

```
1
2
3 Fizz
4
5 Buzz
6 Fizz
7
8
9 Fizz
10 Buzz
11
12 Fizz
13
14
15 Fizz
16
17
18 Fizz
19
20 Buzz
```

```
21 Fizz
22
23
24 Fizz
25 Buzz
26
27 Fizz
28
29
30 Fizz
31
32
33 Fizz
34
35 Buzz
36 Fizz
37
38
39 Fizz
40 Buzz
41
42 Fizz
43
44
45 Fizz
46
47
48 Fizz
49
50 Buzz
```

[49]:
```python
def fizzBuzz(r):
    for i in range(1,51):
        if(i % 3 == 0 and i % 5 ==0):
            print(str(i)+" Fizz Buzz")
        else:
            if(i %3 == 0):
                print(str(i)+" Fizz")
            else:
                if(i % 5 == 0):
                    print(str(i)+" Buzz")
                else:
                    print(str(i))

fizzBuzz(51)
```

```
1
2
```

```
3 Fizz
4
5 Buzz
6 Fizz
7
8
9 Fizz
10 Buzz
11
12 Fizz
13
14
15 Fizz Buzz
16
17
18 Fizz
19
20 Buzz
21 Fizz
22
23
24 Fizz
25 Buzz
26
27 Fizz
28
29
30 Fizz Buzz
31
32
33 Fizz
34
35 Buzz
36 Fizz
37
38
39 Fizz
40 Buzz
41
42 Fizz
43
44
45 Fizz Buzz
46
47
48 Fizz
49
50 Buzz
```

[15]:
```
estimates =␣
↪[1,2,1,24,4,5,357,34,2787,4563,352,343,577,686,857,578,435,212,33,23,343,42,423,54,23,45,65
estimates.sort()
print(estimates)
```

```
[1, 1, 2, 3, 4, 4, 5, 6, 6, 6, 7, 11, 23, 23, 24, 24, 32, 33, 34, 35, 35, 42,
45, 46, 47, 54, 57, 59, 64, 65, 67, 68, 212, 223, 235, 343, 343, 345, 352, 356,
356, 357, 423, 435, 456, 577, 578, 674, 686, 857, 872, 965, 2341, 2584, 2787,
3563, 4326, 4563, 5783, 7658]
```

[16]:
```
for i in range(len(estimates)):
    print(estimates[i])
```

```
1
1
2
3
4
4
5
6
6
6
7
11
23
23
24
24
32
33
34
35
35
42
45
46
47
54
57
59
64
65
67
68
212
223
```

```
235
343
343
345
352
356
356
357
423
435
456
577
578
674
686
857
872
965
2341
2584
2787
3563
4326
4563
5783
7658
```

[17]:
```python
tv = int(0.1*len(estimates))
estimates = estimates[tv:]
estimates = estimates[:len(estimates)-tv]
print(estimates)
```

```
[5, 6, 6, 6, 7, 11, 23, 23, 24, 24, 32, 33, 34, 35, 35, 42, 45, 46, 47, 54, 57,
59, 64, 65, 67, 68, 212, 223, 235, 343, 343, 345, 352, 356, 356, 357, 423, 435,
456, 577, 578, 674, 686, 857, 872, 965, 2341, 2584]
```

[18]:
```python
from statistics import mean
print(mean(estimates))
```

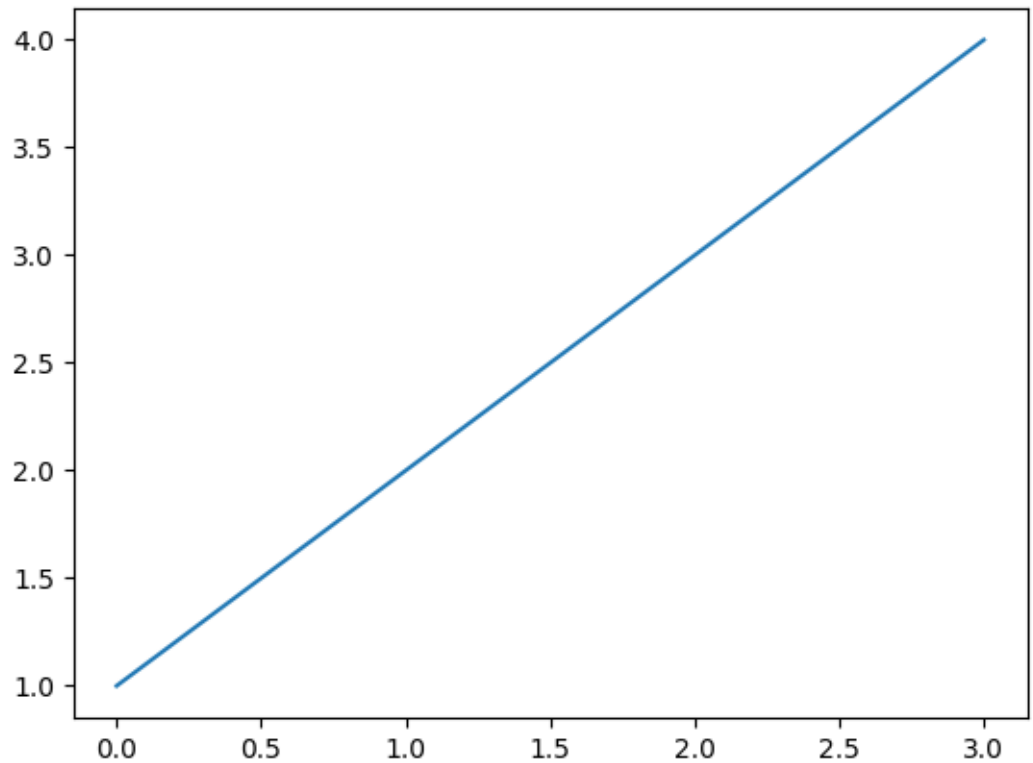
```
322.6666666666667
```

[19]:
```python
from scipy import stats
m = stats.trim_mean(estimates,0.1)
print(m)
```

```
217.575
```

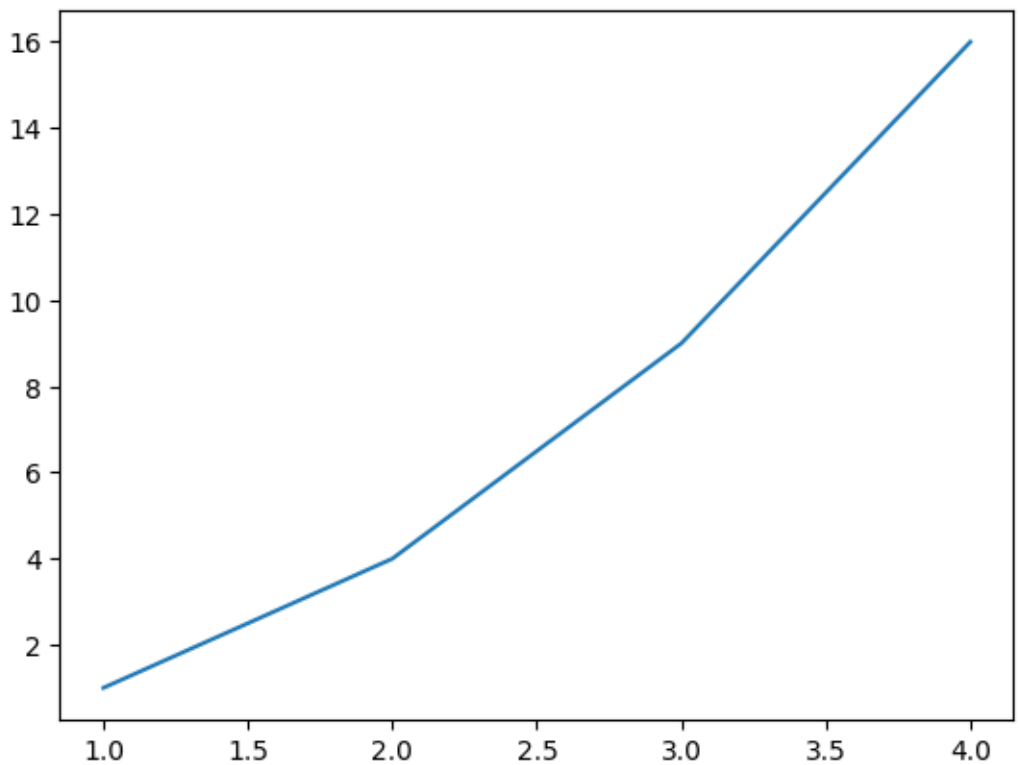
[21]:
```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4])
```

[21]: [<matplotlib.lines.Line2D at 0x7f8e4690ded0>]



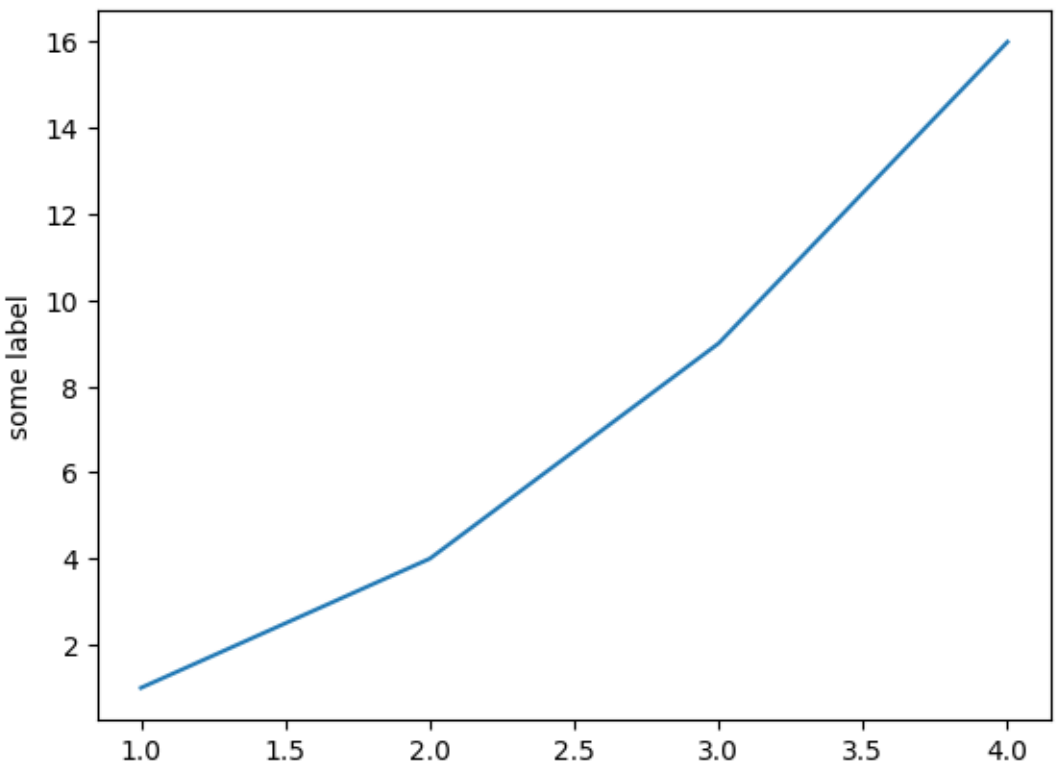
[22]:
```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4],[1,4,9,16])
```

[22]: [<matplotlib.lines.Line2D at 0x7f8e48ad64d0>]



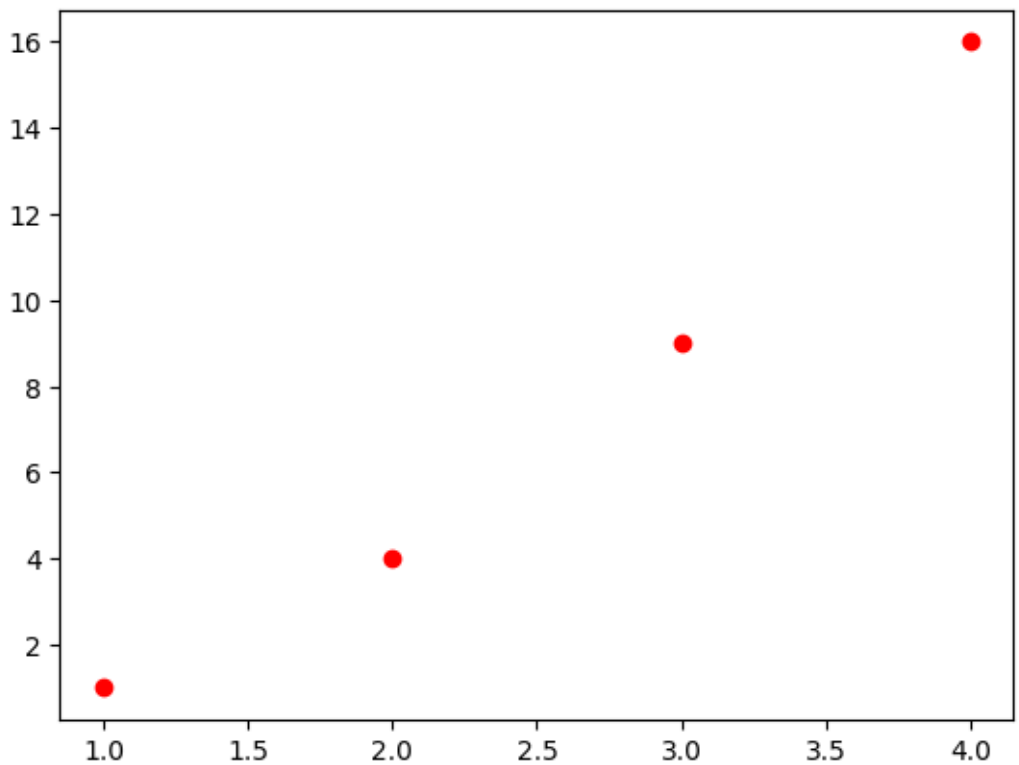
[24]:
```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4],[1,4,9,16])
plt.ylabel("some label")
```

[24]: Text(0, 0.5, 'some label')



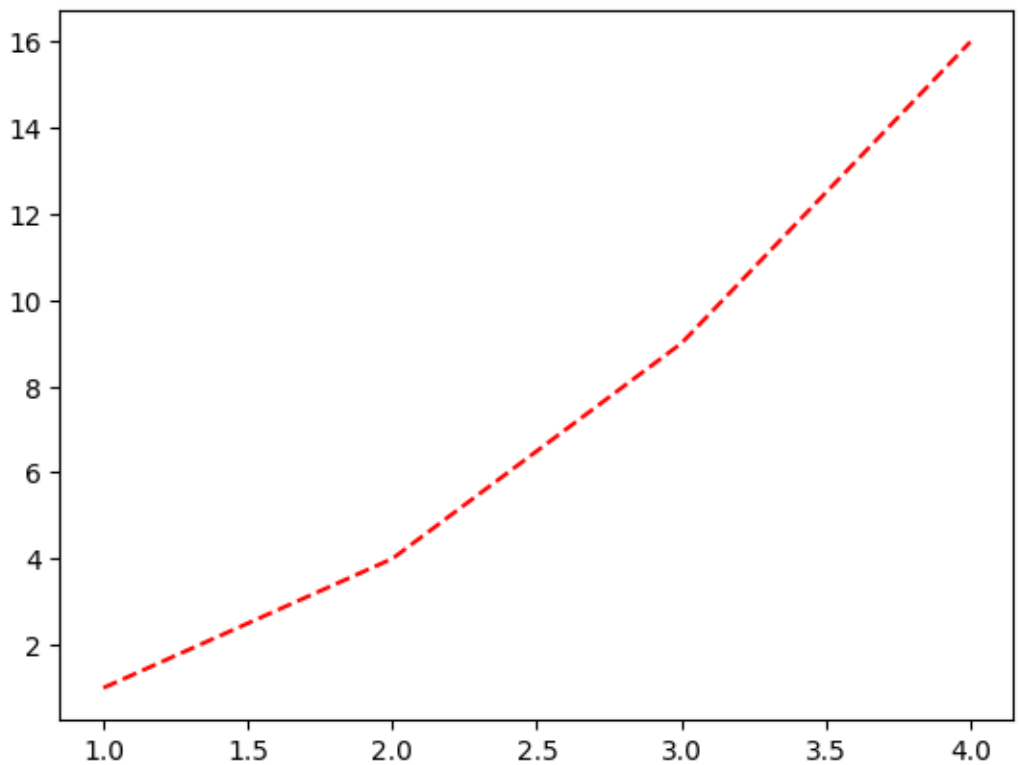
[25]:
```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4],[1,4,9,16],'ro')
```

[25]: [<matplotlib.lines.Line2D at 0x7f8e46701660>]



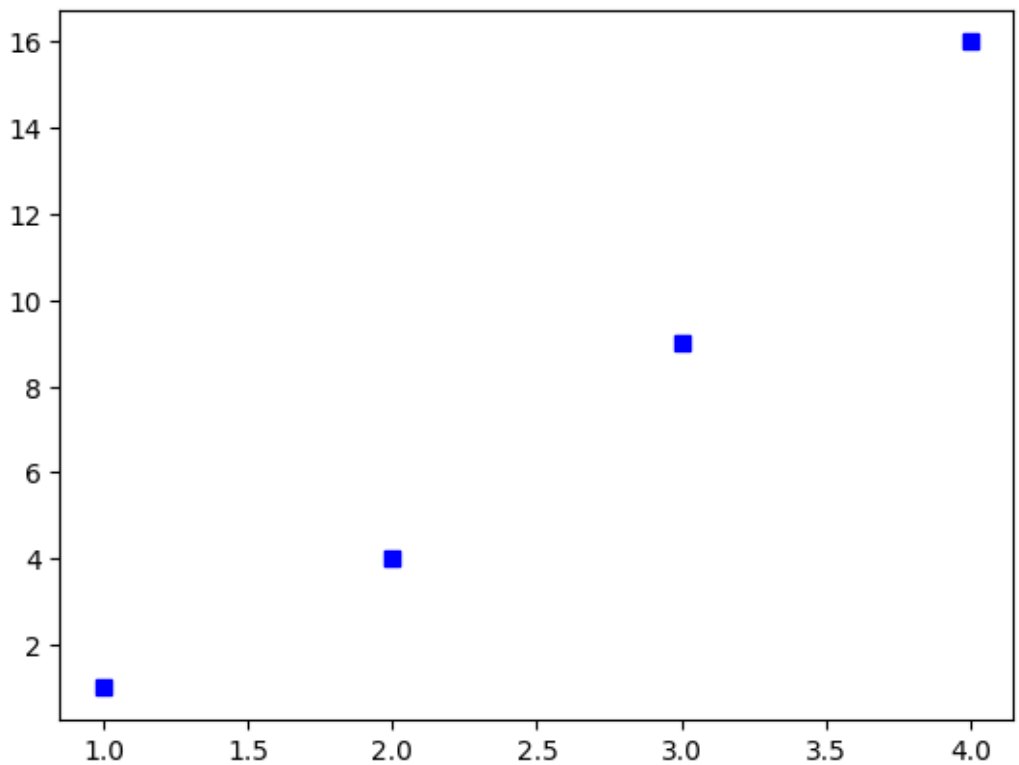
[27]:
```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4],[1,4,9,16],'r--')
```

[27]: [<matplotlib.lines.Line2D at 0x7f8e46798610>]



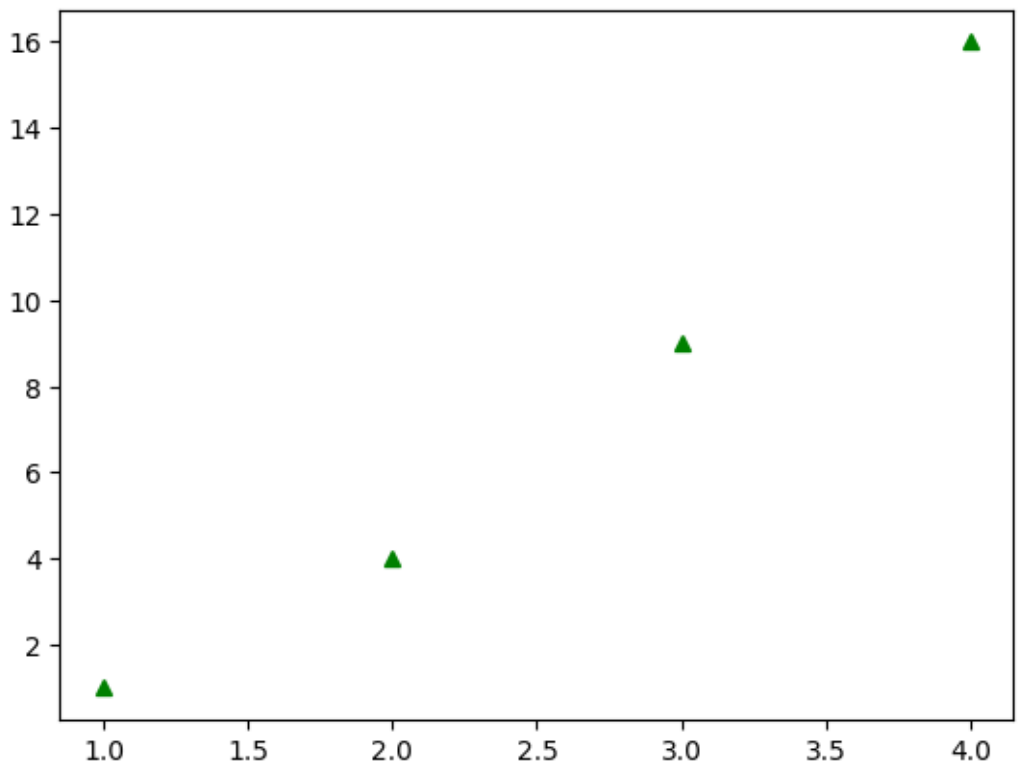
[28]:
```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4],[1,4,9,16],'bs')
```

[28]: [<matplotlib.lines.Line2D at 0x7f8e467f3d90>]



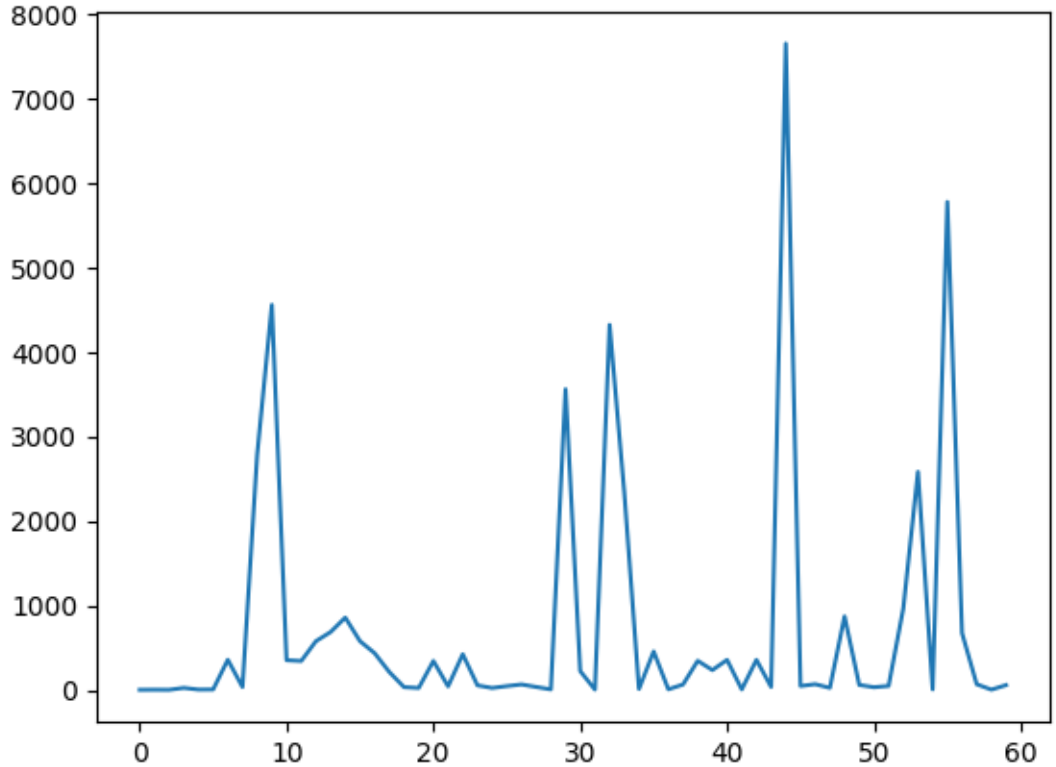
[29]:
```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4],[1,4,9,16],'g^')
```

[29]: [<matplotlib.lines.Line2D at 0x7f8e4667ec80>]



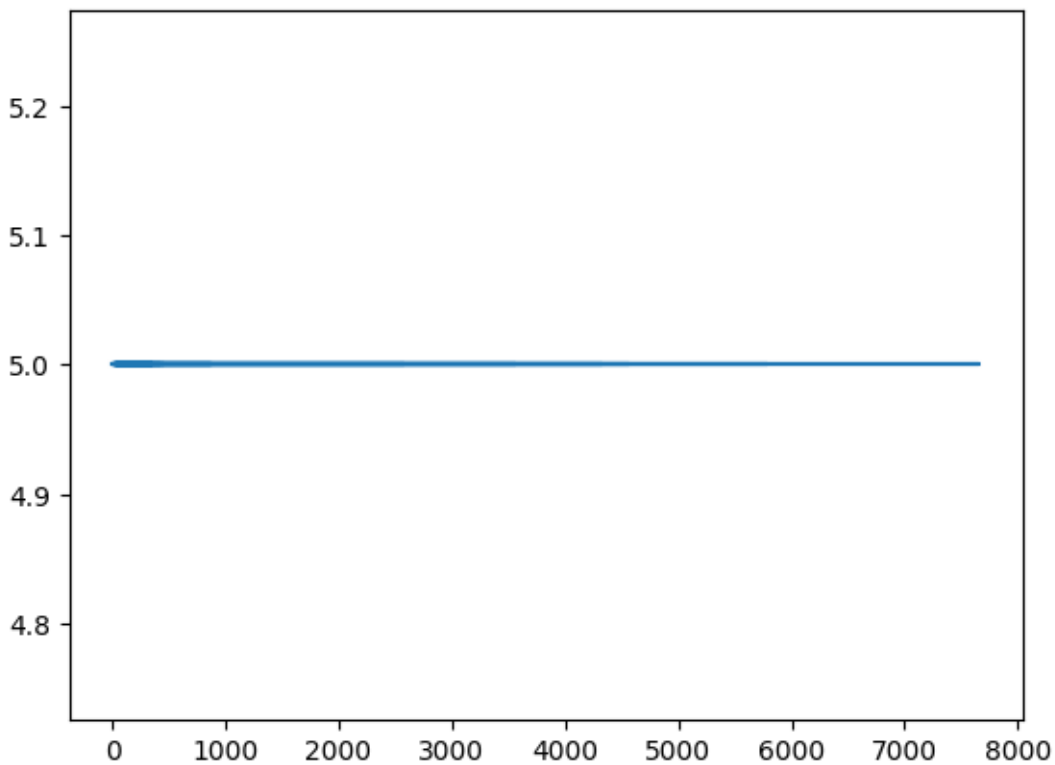
[30]:
```
import matplotlib.pyplot as plt
estimates =␣
↪[1,2,1,24,4,5,357,34,2787,4563,352,343,577,686,857,578,435,212,33,23,343,42,423,54,23,45,65
plt.plot(estimates)
```

[30]: [<matplotlib.lines.Line2D at 0x7f8e4650a560>]



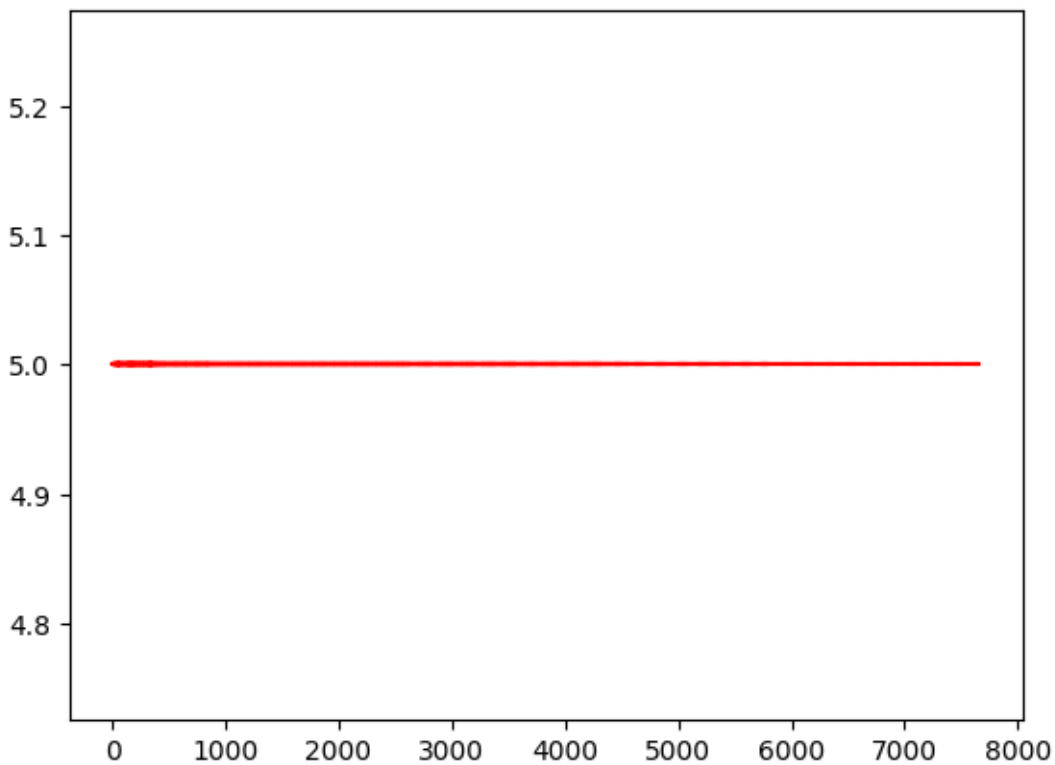
[32]:
```
import matplotlib.pyplot as plt
estimates =␣
 ↪[1,2,1,24,4,5,357,34,2787,4563,352,343,577,686,857,578,435,212,33,23,343,42,423,54,23,45,65
y = []
for i in range(len(estimates)):
    y.append(5)
plt.plot(estimates,y)
```

[32]: [<matplotlib.lines.Line2D at 0x7f8e469c2cb0>]



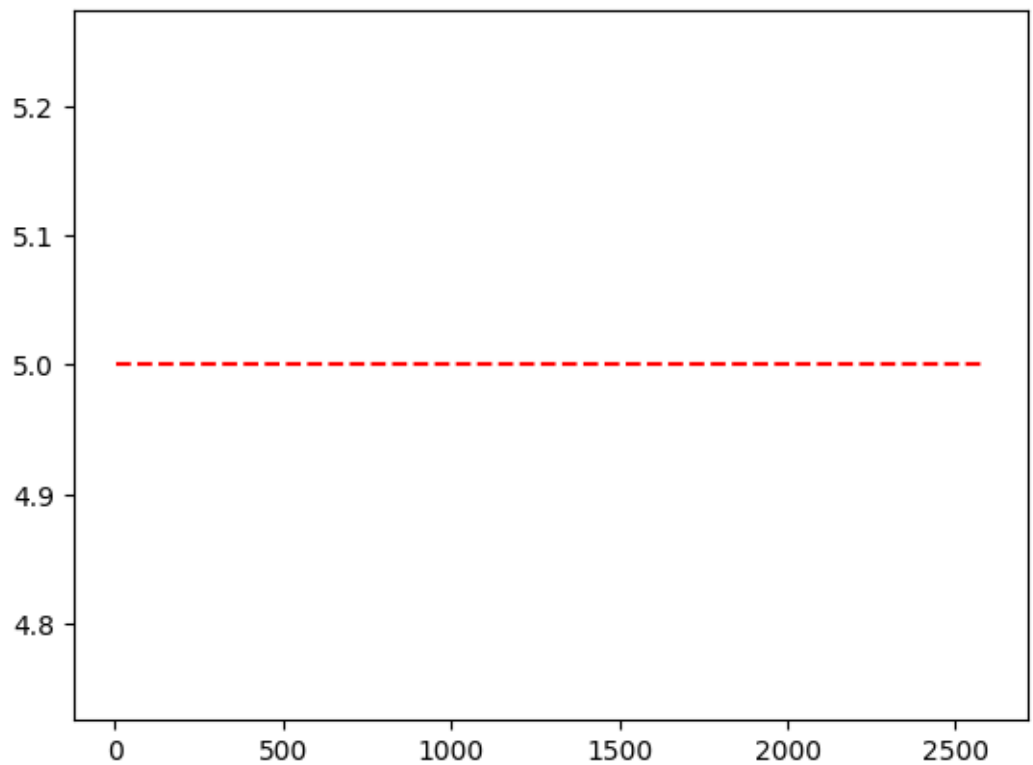
[33]:
```
import matplotlib.pyplot as plt
estimates =␣
↪[1,2,1,24,4,5,357,34,2787,4563,352,343,577,686,857,578,435,212,33,23,343,42,423,54,23,45,65
y = []
for i in range(len(estimates)):
    y.append(5)
plt.plot(estimates,y,'r')
```

[33]: [<matplotlib.lines.Line2D at 0x7f8e466242e0>]



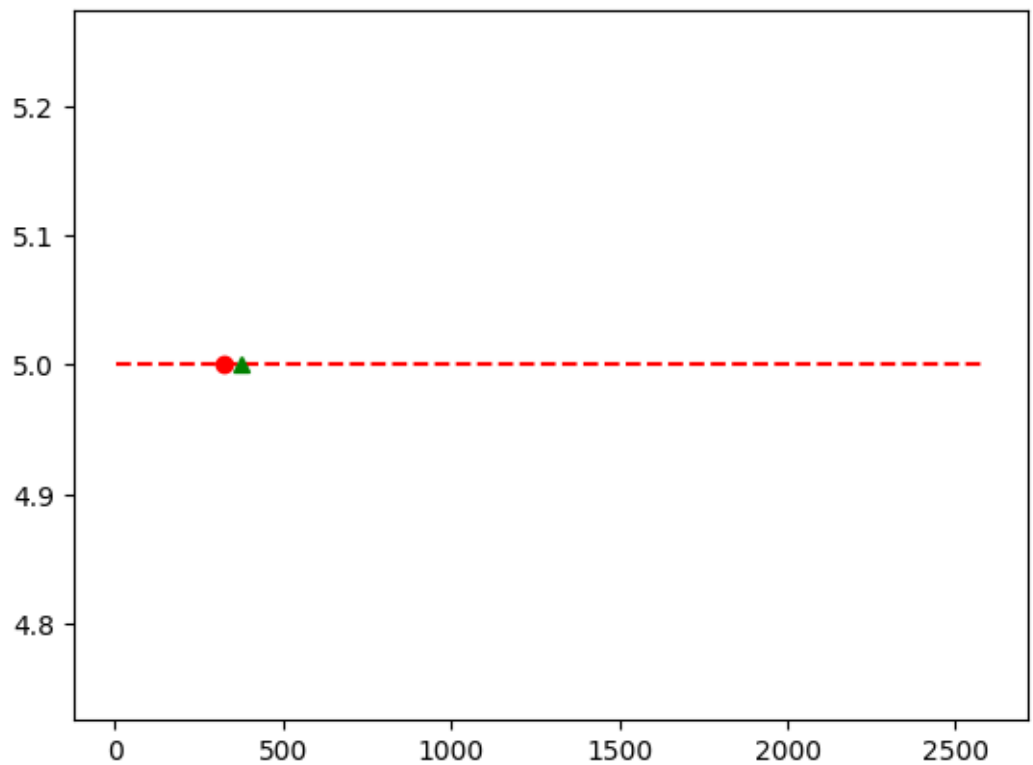
[35]:
```
import matplotlib.pyplot as plt
estimates =␣
↪[1,2,1,24,4,5,357,34,2787,4563,352,343,577,686,857,578,435,212,33,23,343,42,423,54,23,45,65
y = []
estimates.sort()
tv = int(0.1*len(estimates))
estimates = estimates[tv:]
estimates = estimates[:len(estimates)-tv]
for i in range(len(estimates)):
    y.append(5)
plt.plot(estimates,y,'r--')
```

[35]: [<matplotlib.lines.Line2D at 0x7f8e44e29ab0>]



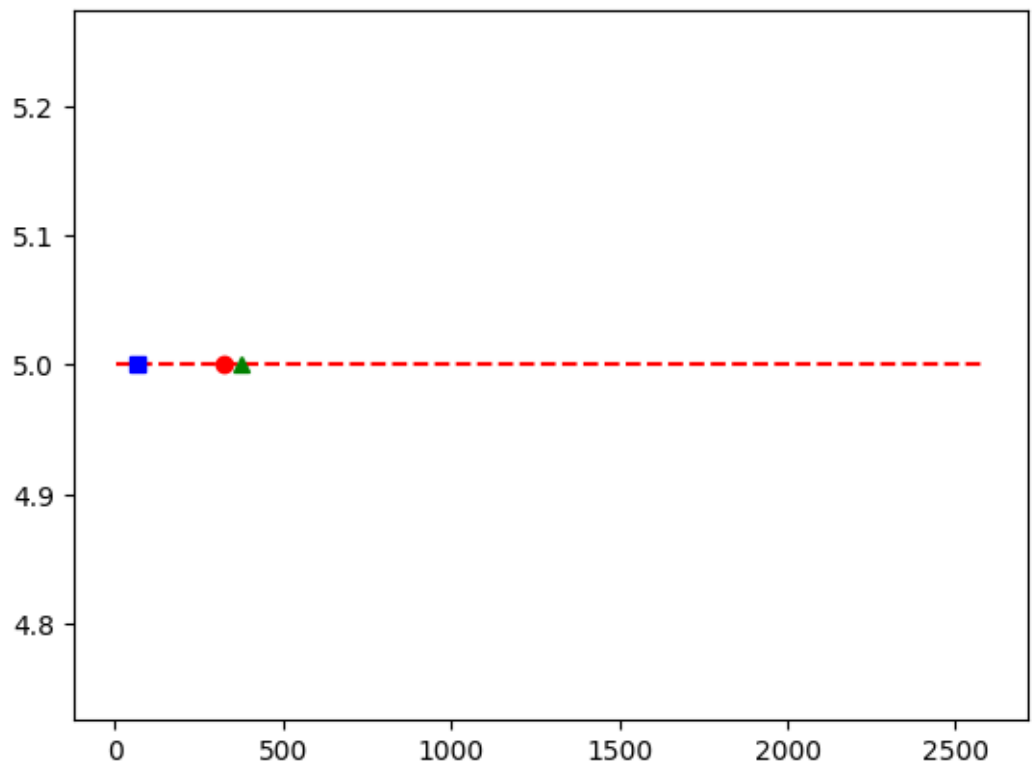
[37]:
```
import statistics
import matplotlib.pyplot as plt
estimates =␣
 ↪[1,2,1,24,4,5,357,34,2787,4563,352,343,577,686,857,578,435,212,33,23,343,42,423,54,23,45,65
y = []
estimates.sort()
tv = int(0.1*len(estimates))
estimates = estimates[tv:]
estimates = estimates[:len(estimates)-tv]
for i in range(len(estimates)):
    y.append(5)
plt.plot(estimates,y,'r--')
plt.plot([statistics.mean(estimates)],[5],'ro')
plt.plot([375],[5],'g^')
```

[37]: [<matplotlib.lines.Line2D at 0x7f8e44d19ae0>]



[40]:
```
import statistics
import matplotlib.pyplot as plt
estimates =␣
 ↪[1,2,1,24,4,5,357,34,2787,4563,352,343,577,686,857,578,435,212,33,23,343,42,423,54,23,45,65
y = []
estimates.sort()
tv = int(0.1*len(estimates))
estimates = estimates[tv:]
estimates = estimates[:len(estimates)-tv]
for i in range(len(estimates)):
    y.append(5)
plt.plot(estimates,y,'r--')
plt.plot([statistics.mean(estimates)],[5],'ro')
plt.plot([statistics.median(estimates)],[5],'bs')
plt.plot([375],[5],'g^')
```

[40]: [<matplotlib.lines.Line2D at 0x7f8e44c7d0c0>]



0.2 jumbled word -comninations permutations

[5]:
```
import random

def choose():
    words␣
 ↪=['rainbow','computer','science','mathematics','player','condition','water','board']
    pick = random.choice(words)
    return pick

def jumble(word):
    jumbled = "".join(random.sample(word,len(word)))
    return jumbled

def thank(p1,p2,pp1,pp2):
    print(p1,"Your score:",pp1)
    print(p2,"Your score:",pp2)

def play():
    p1 = input("Player 1,Enter ur name")
    p2 = input("Player 2,Enter ur name")
    pp1 = 0
```

```
    pp2 = 0
    turn = 0
    while(1):
        #computer task
        pick_word = choose()
        #create question
        q = jumble(pick_word)
        print(q)
        #player 1
        if turn % 2 == 0:
            print(p1,"your turn")
            ans = input("whats on my mind?:")
            if ans == pick_word:
                pp1 = pp1+1
                print("your score :",pp1)
            else:
                print("Better luck next time,word is:",pick_word)
            c = int(input("Press 1 to cont and 0 to quit:"))
            if c == 0:
                thank(p1,p2,pp1,pp2)
                break

        #player 2
        else:
            print(p2,"your turn")
            ans = input("whats on my mind?:")
            if ans == pick_word:
                pp2 = pp2+1
                print("your score :",pp2)
            else:
                print("Better luck next time,word is:",pick_word)
            c = int(input("Press 1 to cont and 0 to quit:"))
            if c == 0:
                thank(p1,p2,pp1,pp2)
                break
        turn = turn + 1
play()
```

```
Player 1,Enter ur name sneha
Player 2,Enter ur name sai

icsneec
sneha your turn

whats on my mind?: science

your score : 1

Press 1 to cont and 0 to quit: 0
```

```
sneha Your score: 1
sai Your score: 0
```

[8]:
```python
import random
movies=["bahubali","temper","athadu","pokiri","avengers","gabbarsingh","kushi","pushpa","kalki
def secretlist(qn):
    secrete=[]
    for i in range(0,len(qn)+1):
        secrete=["_"]*i
    return secrete
def guessletter(qn,secrete):
    #print(qn)
    for i in secrete:
        print(i,end="")
    print()
    letter=input("enter letter: ")
    for i in range(0,len(secrete)):
        if(letter==qn[i]):
            secrete[i]=letter
    for i in secrete:
        print(i,end="")
    print()
    temp=int(input("enter 1 to enter movie name and 0 to guess letter: "))
    if (temp==0):
        guessletter(qn,secrete)
    else:
        guessname(qn)
def guessname(og):
    moviename=input("enter name: ")
    if(og==moviename):
        print("yes you are correct")
    else:
        print("sorry wrong answer")
def play():
    p1=input("enter player1 name: ")
    p2=input("enter player2 name: ")
    turn=0
    intrest=True
    while(intrest):
        qnmovie=random.choice(movies)
        secrete=secretlist(qnmovie)
        if(turn%2==0):
            print(p1,"your turn")
            #print(qnmovie)
        else:
            print(p2,"your turn")
            #print(qnmovie)
```

```
        guessletter(qnmovie,secrete)
        intrst=int(input("enter 1 to continue and 0 to stop: "))
        if(intrst==1):
            intrest=True
        elif(intrst==0):
            intrest=False
        turn+=1
play()
```

```
enter player1 name:  sneha
enter player2 name:  sai

sneha your turn

--------

enter letter:  q

--------

enter 1 to enter movie name and 0 to guess letter:  0

--------

enter letter:  z

--------

enter 1 to enter movie name and 0 to guess letter:  1
enter name:  saneha

sorry wrong answer

enter 1 to continue and 0 to stop:  0
```

[1]:
```
import random
movies=["jungle book","sitaramam","darling","kalki","andala␣
↪rakshasi","jalsa","anand","sakhi","don","damaka","ekkadiki pothavu␣
↪chinnavada"]
def create(pick):
    word=pick.split()
    for i in range (len(word)):
        word[i]="*"*len(word[i])
    qn=' '.join(word)
    return qn

def present(ch,movie):
    c=movie.count(ch)
    if c==0:
        print("absentttttttt")
        return False
    else:
        print("ttttttt")
```

```
        return True

def unlock(modify,pick,ch):
    ref=list(pick)
    qlist=list(modify)
    l=[]
    for i in range(len(pick)):
        if ref[i]==ch or ref[i]==" ":
            l.append(ref[i])
        else:
            if qlist[i]=="*":
                l.append("*")
            else:
                l.append(ref[i])
    qnew=''.join(l)
    return qnew

def play():
    p1=input("player1 name:")
    p2=input("player2 name:")
    p1score=0
    p2score=0
    turn=0
    p=True
    while p:
        if turn%2==0:
            print(p1,"turn")
            pick=random.choice(movies)
            q=create(pick)
            print(q)
            modify=q
            not_said=True
            while not_said:
                ch=input("enter ch:")
                if(present(ch,pick)):
                #unlock
                    modify=unlock(modify,pick,ch)
                    print(modify)
                    d=int(input("enter 1 to unlock,2 to guess ch"))
                    if d==1:
                        ans=input("enter movie:")
                        if ans==pick:
                            p1score=p1score+1
                            print("correct")
                            not_said=False
                            print(p1,"score:",p1score)
                        else:
```

```
                            print("try again")
                else:
                    print(ch,"not present")
            c=int(input("enter 1 to cont.. 0 to quit"))
            if c==0:
                print(p1,"score is:",p1score)
                print(p2,"score is:",p2score)
                p=False

        else:
            print(p2,"turn")
            pick=random.choice(movies)
            q=create(pick)
            print(q)
            modify=q
            not_said=True
            while not_said:
                ch=input("enter ch:")
                if(present(ch,pick)):
                #unlock
                    modify=unlock(modify,pick,ch)
                    print(modify)
                    d=int(input("enter 1 to unlock,2 to guess ch"))
                    if d==1:
                        ans=input("enter movie:")
                        if ans==pick:
                            p2score=p2score+1
                            print("correct")
                            not_said=False
                            print(p2,"score:",p2score)
                        else:
                            print("try again")
                else:
                    print(ch,"not present")
            c=int(input("enter 1 to cont.. 0 to quit"))
            if c==0:
                print(p1,"score is:",p1score)
                print(p2,"score is:",p2score)
                p=False
        turn=turn+1


play()
```

```
player1 name: sne
player2 name: sa

sne turn
```

```
****** ********

enter ch: a

tttttt
a**a*a *a***a**

enter 1 to unlock,2 to guess ch 0
enter ch: w

absentttttt
w not present

enter ch: e

absentttttt
e not present

enter ch: f

absentttttt
f not present

enter ch: t

absentttttt
t not present

enter ch: u

absentttttt
u not present
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[1], line 106
    102                 p=False
    103         turn=turn+1
--> 106 play()

Cell In[1], line 50, in play()
     48 not_said=True
     49 while not_said:
---> 50     ch=input("enter ch:")
     51     if(present(ch,pick)):
     52     #unlock
     53         modify=unlock(modify,pick,ch)

File ~/.local/lib/python3.10/site-packages/ipykernel/kernelbase.py:1282, in␣
 ↪Kernel.raw_input(self, prompt)
   1280     msg = "raw_input was called, but this frontend does not support␣
 ↪input requests."
   1281     raise StdinNotImplementedError(msg)
```

```
-> 1282 return self._input_request(
   1283     str(prompt),
   1284     self._parent_ident["shell"],
   1285     self.get_parent("shell"),
   1286     password=False,
   1287 )

File ~/.local/lib/python3.10/site-packages/ipykernel/kernelbase.py:1325, in␣
 ↪Kernel._input_request(self, prompt, ident, parent, password)
   1322 except KeyboardInterrupt:
   1323     # re-raise KeyboardInterrupt, to truncate traceback
   1324     msg = "Interrupted by user"
-> 1325     raise KeyboardInterrupt(msg) from None
   1326 except Exception:
   1327     self.log.warning("Invalid Message:", exc_info=True)

KeyboardInterrupt: Interrupted by user
```

0.3 file operations

[1]:
```
with open("jayi.py") as myfile:
    print(myfile.read())
```

```
# print("hello world")
# a = 10
# b= "sneha"
# print(a,b)
# #complex

# n=9
# a=[]
# for i in range(n):
#     l1=[]
#     p=0
#     for j in range(i):
#         l1.append(j)
#     for k in l1:
#         p+=1
#     a.append(p)
#     print(a)
# k=0
# for l in a:
#     k+=1
# print(k)


# n=3
```

```
# k=0
# a=[]
# b=0
# while(n!=0):
#     k=k+(n%10)
#     print("fyfd",k)
#     a.append(n%10)
#     print(a)
#     n=n//10
#     print(n)
# a.sort()
# for i in a:
#     b=b+i
#     print(b)
# if(b==k):
#     print("wis")
# else:
#     print("many")



# def fun1(number):
#     l1=[]
#     for i in range(1,number):
#         if number % i ==0:
#             l1.append(i)
#     return l1
# def fun2(n1,n2):
#     flag =False
#     l2=[]
#     for i in fun1(n1):
#         for j in fun1(n2):
#             if i==j:
#                 flag=True
#                 if flag == True:
#                     print("sneha")
#                 l2.append(i)
#             print(l2)
#     if len(l2)>0:
#         print("compl")
# fun2(2,3)
# print("2,3 done")
# fun2(0,0)
# print("00 dine")
# fun2(1,1)
# print("1,1 done")
# fun2(1,0)
# print("1,0 done")
```

```
import random

# Randomly select a door with a car
ini = random.randint(0, 2)
doors = ['g', 'g', 'c']

# Determine which door Monty opens
for i in range(3):
    if i != ini and doors[i] == 'g':
        m = i
        break

# Determine which door Monty will not open
if  True:
    for i in range(3):
        if i != ini and i != m:
            f = i
            print(f)
            break
else:
    f = m
    print(f)
```

[4]:
```
with open("jayi.py","r") as myfile:
    print(myfile.read())
with open("jayi.py","w") as myfile:
    myfile.write("i am fine")
myfile.close()
```

[5]:
```
with open("jayi.py","r") as myfile:
    print(myfile.read())
```

```
i am fine
```

[6]:
```
with open("jayi.py","r+") as myfile:
    print(myfile.read())
    myfile.write("i am fine")
myfile.close()
```

```
i am fine
```

[1]:
```
with open("jayi.py","r") as myfile:
    print(myfile.read())
```

```
i am finei am fine
```

[2]:
```
data_dna.txt
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 data_dna.txt

NameError: name 'data_dna' is not defined
```

0.4 Magic sqaure

[21]:
```
#magic square
def magic_square(n):
    magicsq=[]
    for i in range(n):
        l=[]
        for j in range(n):
            l.append(0)
        magicsq.append(l)
    # for i in range(3):
    #     #l=[]
    #     for j in range(3):
    #         #l.append(0)
    #         print(magicsq[i][j],end=" ")
    #         print()
    #     #magicsq.append(l)
    #     # e= [0 for i in range(3)]
    #     #f=[[0 for i in rage(3)] for j in range(3)]
    i=n//2
    j=n-1
    num=n*n
    count=1
    while(count<=num):
        if(i==-1 and j==n): #cond 4
            j=n-2
            i=0
        else:
            if(j==n): #col value exceeds
                j=0
            if(i<0): #row becomes -1
```

```
                i=n-1
        if(magicsq[i][j]!=0):
            j=j-2
            i=i+1
            continue
        else:
            magicsq[i][j]=count
            count+=1
        i=i-1
        j=j+1 #cond 1
    for i in range(n):
        for j in range(n):
            print(magicsq[i][j],end=" ")
        print()
magic_square(3)
print()
magic_square(5)
print()
magic_square(7)
```

```
2 7 6
9 5 1
4 3 8

9 3 22 16 15
2 21 20 14 8
25 19 13 7 1
18 12 6 5 24
11 10 4 23 17

20 12 4 45 37 29 28
11 3 44 36 35 27 19
2 43 42 34 26 18 10
49 41 33 25 17 9 1
40 32 24 16 8 7 48
31 23 15 14 6 47 39
22 21 13 5 46 38 30
```

[1]:
```
import string
print(string.ascii_letters)
```

```
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
```

0.5 Doddle game

[15]:
```
import string
import random
symbols=[]
symbols=list(string.ascii_letters)
card1=[0]*5
card2=[0]*5
pos1=random.randint(0,4)
pos2=random.randint(0,4)
print(pos1)
print(pos2)
samesymbol=random.choice(symbols)
symbols.remove(samesymbol)
if pos1==pos2:
    card2[pos1]=samesymbol
    card1[pos1]=samesymbol
else:
    card1[pos1]=samesymbol
    card2[pos2]=samesymbol
    card1[pos2]=random.choice(symbols)
    symbols.remove(card1[pos2])
    card2[pos1]=random.choice(symbols)
    symbols.remove(card2[pos1])
i=0
while(i<5):
    if i!=pos1 and i!=pos2 :
        alphabet1=random.choice(symbols)
        symbols.remove(alphabet1)
        alphabet2=random.choice(symbols)
        symbols.remove(alphabet2)
        card1[i]=alphabet1
        card2[i]=alphabet2
    i=i+1
print(card1)
print(card2)
ch=input("spot similar")
if ch==samesymbol:
    print("right")
else:
    print("wrong")
```

```
3
0
['y', 's', 'H', 'i', 'C']
['i', 'R', 'X', 'G', 'E']
```

```
spot similar i
```

right

0.6 Find ur twin

[42]:
```
import random
year=random.randint(1993,2018)
if (year%4==0 and year%100!=0 or year%400==0):
    print("leap year")
else:
    print("non leap year")
print(year)
```

```
leap year
2004
```

0.7 Birthday Paradox

[45]:
```
import random
import datetime
birthday=[]
i=0
while(i<50):
    year=random.randint(1895,2024)
    if (year%4==0 and year%100!=0 or year%400==0):
        leap=1
    else:
        leap=0
    month=random.randint(1,12)
    if month==2 and leap==1:
        day=random.randint(1,29)
    elif month==2and leap==0:
        day=random.randint(1,28)
    elif month==7 or month==8:
        day=random.randint(1,31)
    elif month%2!=0 and month<7:
        day=random.randint(1,31)
    elif month%2!=0 and month>7 and month<12:
        day=random.randint(1,31)
    else:
        day=random.randint(1,30)
    dd=datetime.date(year,month,day)
    day_of_year=dd.timetuple().tm_yday
    i=i+1
    birthday.append(day_of_year)
birthday.sort()
i=0
while(i<50):
    print(birthday[i])
```

```
    i=i+1
```

```
2
7
10
18
22
25
31
36
44
45
59
73
74
91
94
95
98
104
107
121
124
131
136
140
186
189
189
193
204
214
217
223
227
231
239
243
256
261
275
281
281
310
316
320
327
```

```
328
333
342
343
353
```

[46]:
```
import datetime
print(datetime.date.today())
```

```
2024-09-03
```

[52]:
```
print(datetime.date.today().strftime("%Y"))
print(datetime.date.today().strftime("%B"))
print(datetime.date.today().strftime("%d"))
```

```
2024
September
03
```

[59]:
```
print("week no. of month",datetime.date.today().strftime("%W"))
print("weekday of week",datetime.date.today().strftime("%w"))
print("day of year",datetime.date.today().strftime("%j"))
print("day of week",datetime.date.today().strftime("%A"))
```

```
week no. of month 36
weekday of week 2
day of year 247
day of week Tuesday
```

[60]:
```
datetime.datetime.now()
```

[60]:
```
datetime.datetime(2024, 9, 3, 22, 14, 39, 533330)
```

0.8 Favourite movie

[ ]:
```
import random
list=["junglebook","sitaramam","darling","kalki","andalarakshasi","jalsa"]
mov=random.choice(list)
print(mo
guess="*"*len(mov)
c=input("enter letter")
for i in mov:
    if c==i:
        guess[i]=mov[i]
    else:
        c=input("enter letter")
```

[6]:
```
import random
movies=["jungle book","sitaramam","darling","kalki","andala␣
↪rakshasi","jalsa","anand","sakhi","don","damaka","ekkadiki pothavu␣
↪chinnavada"]

def create(movie):
    n=len(movie)
    letters=list(movie)
    temp=[]
    for i in range(n):
        if letters[i]==" ":
            temp.append(" ")
        else:
            temp.append('*')
    qn=" ".join(str(x) for x in temp)
    return qn

def is_present(letter,movie):
    c=movie.count(letter)
    if c==0:
        return False
    else:
        return True

def unlock(qn,movie,letter):
    ref=list(movie)
    qlist=list(qn)
    temp=[]
    n=len(movie)
    for i in range(n):
        if ref[i]==" " or ref[i]==letter:
            temp.append(ref[i])
        else:
            if qlist[i]=="*":
                temp.append("*")
            else:
                temp.append(ref[i])
    qnew=" ".join(str(x) for x in temp)
    return qnew


def play():
    p1=input("player1 name:")
    p2=input("player2 name:")
    p1score=0
    p2score=0
    turn=0
```

```
    willing=True
    while willing:
        if turn%2==0:
            print(p1," turn")
            picked_movie=random.choice(movies)
            qn=create(picked_movie)
            print(qn)
            modified=qn
            not_said=True
            while not_said:
                letter=input("enter letter:")
                if (is_present(letter,picked_movie)):
                    #unlock
                    modified=unlock(modified,picked_movie,letter)
                    print(modified)
                    d=int(input("enter 1 to unlock movie & 2 to unlock letter:
↪"))
                    if d==1:
                        ans=input("enter movie name:")
                        if ans==picked_movie:
                            p1score=p1score+1
                            print("it's correct")
                            not_said=False
                            print(p1,"score is:",p1score)
                        else:
                            print("sorry its wrong")
                else:
                    print(letter," is not present")
            c=int(input("enter 1 to cont.. 0 to quit"))
            if c==0:
                print(p1,"score is:",p1score)
                print(p2,"score is:",p2score)
                willing=False

        else:
            print(p2," turn")
            picked_movie=random.choice(movies)
            qn=create(picked_movie)
            print(qn)
            modified=qn
            not_said=True
            while not_said:
                letter=input("enter letter:")
                if (is_present(letter,picked_movie)):
                    #unlock
                    modified=unlock(modified,picked_movie,letter)
                    print(modified)
```

```
                    d=int(input("enter 1 to unlock movie & 2 to unlock letter:
↪"))
                    if d==1:
                        ans=input("enter movie name:")
                        if ans==picked_movie:
                            p2score=p1score+1
                            print("it's correct")
                            not_said=False
                            print(p2,"score is:",p2score)
                        else:
                            print("sorry its wrong")
                else:
                    print(letter," is not present")
            c=int(input("enter 1 to cont.. 0 to quit"))
            if c==0:
                print(p1,"score is:",p1score)
                print(p2,"score is:",p2score)
                willing=False

        turn=turn+1

play()
```

```
player1 name: s
player2 name: v

s  turn
* * * * *

enter letter: a

* a * s a

enter 1 to unlock movie & 2 to unlock letter: j
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[6], line 112
    108                 willing=False
    110         turn=turn+1
--> 112 play()

Cell In[6], line 61, in play()
     59 modified=unlock(modified,picked_movie,letter)
     60 print(modified)
---> 61 d=int(input("enter 1 to unlock movie & 2 to unlock letter:"))
     62 if d==1:
     63     ans=input("enter movie name:")
```

```
ValueError: invalid literal for int() with base 10: 'j'
```

[8]:
```
import random
movies=["jungle book","sitaramam","darling","kalki","andala␣
↪rakshasi","jalsa","anand","sakhi","don","damaka","ekkadiki pothavu␣
↪chinnavada"]
def create(pick):
    word=pick.split()
    for i in range (len(word)):
        word[i]="*"*len(word[i])
    qn=' '.join(word)
    return qn

def present(ch,movie):
    c=movie.count(ch)
    if c==0:
        return False
    else:
        return True

def unlock(modify,pick,ch):
    ref=list(pick)
    qlist=list(modify)
    l=[]
    for i in range(len(pick)):
        if ref[i]==ch or ref[i]==" ":
            l.append(ref[i])
        else:
            if qlist[i]=="*":
                l.append("*")
            else:
                l.append(ref[i])
    qnew=''.join(l)
    return qnew

def play():
    p1=input("player1 name:")
    p2=input("player2 name:")
    p1score=0
    p2score=0
    turn=0
    p=True
    while p:
        if turn%2==0:
            print(p1,"turn")
            pick=random.choice(movies)
            q=create(pick)
```

```
        print(q)
        modify=q
        not_said=True
        while not_said:
            ch=input("enter ch:")
            if(present(ch,pick)):
            #unlock
                modify=unlock(modify,pick,ch)
                print(modify)
                d=int(input("enter 1 to unlock,2 to guess ch"))
                if d==1:
                    ans=input("enter movie:")
                    if ans==pick:
                        p1score=p1score+1
                        print("correct")
                        not_said=False
                        print(p1,"score:",p1score)
                    else:
                        print("try again")
            else:
                print(ch,"not present")
        c=int(input("enter 1 to cont.. 0 to quit"))
        if c==0:
            print(p1,"score is:",p1score)
            print(p2,"score is:",p2score)
            p=False

    else:
        print(p2,"turn")
        pick=random.choice(movies)
        q=create(pick)
        print(q)
        modify=q
        not_said=True
        while not_said:
            ch=input("enter ch:")
            if(present(ch,pick)):
            #unlock
                modify=unlock(modify,pick,ch)
                print(modify)
                d=int(input("enter 1 to unlock,2 to guess ch"))
                if d==1:
                    ans=input("enter movie:")
                    if ans==pick:
                        p2score=p2score+1
                        print("correct")
                        not_said=False
```

```
                            print(p2,"score:",p2score)
                        else:
                            print("try again")
                else:
                    print(ch,"not present")
            c=int(input("enter 1 to cont.. 0 to quit"))
            if c==0:
                print(p1,"score is:",p1score)
                print(p2,"score is:",p2score)
                p=False
        turn=turn+1


play()
```

```
player1 name: sai
player2 name: sneha

sai turn
****** ****

enter ch: b

****** b***

enter 1 to unlock,2 to guess ch 2
enter ch: a

a not present

enter ch: j

j***** b***

enter 1 to unlock,2 to guess ch 2
enter ch: o

j***** boo*

enter 1 to unlock,2 to guess ch 1
enter movie: jungle book

correct
sai score: 1

enter 1 to cont.. 0 to quit 1

sneha turn
***

enter ch: a

a not present

enter ch: o
```

```
*o*

enter 1 to unlock,2 to guess ch 2
enter ch: n

*on

enter 1 to unlock,2 to guess ch 2
enter ch: d

don

enter 1 to unlock,2 to guess ch 1
enter movie: don

correct
sneha score: 1

enter 1 to cont.. 0 to quit 1

sai turn
*****

enter ch: k

**k**

enter 1 to unlock,2 to guess ch 2
enter ch: s

s*k**

enter 1 to unlock,2 to guess ch 1
enter movie: sikil

try again

enter ch: s

s*k**

enter 1 to unlock,2 to guess ch 1
enter movie: sakhi

correct
sai score: 2

enter 1 to cont.. 0 to quit 1

sneha turn
******** ******* **********

enter ch: a

***a**** ****a** *****a*a*a

enter 1 to unlock,2 to guess ch 2
enter ch: e

e**a**** ****a** *****a*a*a
```

```
enter 1 to unlock,2 to guess ch 2
enter ch: k

ekka**k* ****a** *****a*a*a

enter 1 to unlock,2 to guess ch 2
enter ch: d

ekkad*k* ****a** *****a*ada

enter 1 to unlock,2 to guess ch 2
enter ch: i

ekkadiki ****a** **i**a*ada

enter 1 to unlock,2 to guess ch 2
enter ch: p

ekkadiki p***a** **i**a*ada

enter 1 to unlock,2 to guess ch 2
enter ch: o

ekkadiki po**a** **i**a*ada

enter 1 to unlock,2 to guess ch 1
enter movie: ekkadiki pothavu chinnavada

correct
sneha score: 2

enter 1 to cont.. 0 to quit 0

sai score is: 2
sneha score is: 2
```

[1]:
```
import random
movies␣
↪=["avengers","marvel","thor","increadiblehulk","captainamerica","ironman","drishyam","anand

def create_question(movie):
    n = len(movie)
    letters=list(movie)
    temp=[]
    for i in range(n):
        if letters[i]==' ':
            temp.append(' ')
        else:
            temp.append('*')
    qn=''.join(str(x) for x in temp)
    return qn

def is_present(letter,movie):
    c=movie.count(letter)
```

```
    if(c==0):
        return False
    else:
        return True
    print("present")

def unlock(qn,movie,letter):
    ref=list(movie)
    qn_list=list(qn)
    temp=[]
    n=len(movie)
    for i in range(n):
        if ref[i]==' ' or ref[i]==letter:
            temp.append(ref[i])
        else:
            if qn_list[i]=='*':
                temp.append('*')
            else:
                temp.append(ref[i])
    qn_new=''.join(str(x) for x in temp)
    return qn_new


def play():
    p1name=input('Player 1! Please enter your name: ')
    p2name=input('Player 2! Please enter your name: ')
    pp1=0
    pp2=0
    turn=0
    willing =True
    while willing:
        if turn%2==0:
            #player1
            print(p1name,' Your turn ')
            picked_movie=random.choice(movies)
            qn=create_question(picked_movie)
            print(qn)
            modified_qn=qn
            not_said=True
            while not_said:
                letter=input('Your letter: ')
                if(is_present(letter,picked_movie)):

                    #unlock
                    modified_qn=unlock(modified_qn,picked_movie,letter)
                    print(modified_qn)
```

```
                    d=int(input('Press 1 to guess the movie 0 to unlock another␣
↪letter'))
                    if d==1:
                        ans=input('Your answer: ')
                        if ans == picked_movie:
                            pp1 = pp1+1
                            print(' Correct ')
                            not_said=False
                            print(p1name,'Your score : ',pp1)
                        else:
                            print('Wrong answer. Try again.')
                else:
                    print(letter,' not found ')
            c=int(input('Press 1 to continue or 0 to quit'))
            if c==0:
                print(p1name,' Your score : ',pp1)
                print(p2name,' Your score : ',pp2)
                print('Thanks foe playing')
                print('Have a nice day.')
                willing=False

        else:
            #player 2
            print(p2name,' Your turn ')
            picked_movie=random.choice(movies)
            qn=create_question(picked_movie)
            print(qn)
            modified_qn=qn
            not_said=True
            while not_said:
                letter=input('Your letter: ')
                if(is_present(letter,picked_movie)):

                    #unlock
                    modified_qn=unlock(modified_qn,picked_movie,letter)
                    print(modified_qn)
                    d=int(input('Press 1 to guess the movie 0 to unlock another␣
↪letter'))
                    if d==1:
                        ans=input('Your answer: ')
                        if ans == picked_movie:
                            pp1 = pp1+1
                            print(' Correct ')
                            not_said=False
                            print(p1name,'Your score : ',pp2)
                        else:
                            print('Wrong answer. Try again.')
```

```
                else:
                    print(letter,' not found ')
            c=int(input('Press 1 to continue or 0 to quit'))
            if c==0:
                print(p1name,' Your score : ',pp1)
                print(p2name,' Your score : ',pp2)
                print('Thanks foe playing')
                print('Have a nice day.')
                willing=False
        turn = turn+1
play()
```

```
Player 1! Please enter your name:  sneh
Player 2! Please enter your name:  amar

sneh  Your turn
***************

Your letter:  a

*****a*********

Press 1 to guess the movie 0 to unlock another letter 0
Your letter:  c

**c**a*********

Press 1 to guess the movie 0 to unlock another letter 0
Your letter:  p

p  not found

Your letter:  h

**c**a*****h***

Press 1 to guess the movie 0 to unlock another letter 1
Your answer:  incrediblehulk

Wrong answer. Try again.

Your letter:  e

**c*ea****eh***

Press 1 to guess the movie 0 to unlock another letter 0
Your letter:  i

i*c*ea*i**eh***

Press 1 to guess the movie 0 to unlock another letter 0
Your letter:  n

inc*ea*i**eh***
```

```
Press 1 to guess the movie 0 to unlock another letter 0
Your letter:  r

increa*i**eh***

Press 1 to guess the movie 0 to unlock another letter 1
Your answer:  increadiblehulk

 Correct
sneh Your score :  1

Press 1 to continue or 0 to quit 1

amar  Your turn
*****

Your letter:  a

a*a**

Press 1 to guess the movie 0 to unlock another letter 1
Your answer:  anand

 Correct
sneh Your score :  0

Press 1 to continue or 0 to quit 0

sneh  Your score :  2
amar  Your score :  0
Thanks foe playing
Have a nice day.
```

0.9 Dictionary

[9]:
```
con_factor={}
con_factor
```

[9]: {}

[10]:
```
con_factor['dollar']=83
con_factor
```

[10]: {'dollar': 83}

[11]:
```
con_factor['euro']=80
con_factor
```

[11]: {'dollar': 83, 'euro': 80}

[12]:
```
con_factor['euro']
```

[12]: 80

[16]:
```
con_factor.keys()
```

[16]: dict_keys(['dollar', 'euro'])

[17]:
```
print(list(con_factor.keys()))
```

```
['dollar', 'euro']
```

[18]:
```
con_factor.values()
```

[18]: dict_values([83, 80])

[19]:
```
con_factor.items()
```

[19]: dict_items([('dollar', 83), ('euro', 80)])

[20]:
```
#update
con_factor['dollar']=45
con_factor
```

[20]: {'dollar': 45, 'euro': 80}

[21]:
```
del con_factor['euro']
con_factor
```

[21]: {'dollar': 45}

[24]:
```
d=5
r=5*con_factor['dollar']
r
```

[24]: 225

0.10 speech recognisation

[28]:
```
import speech_recognition as sr
audio=("complex_tone.wav")
r=sr.Recognizer()
with sr.AudioFile(audio) as source:
    a=r.record(source)
#reads audio
try:
    print("audio"+r.recognize_google(a))
except sr.UnknownValueError:
    print("google speech recognisation cant understand")
except sr.RequestError:
    print("couldnt get from google")
```

google speech recognisation cant understand

0.11 doors and a twist

[ ]:
```
import random
doors=[0]*3
goatdoor=[0,0]
swap=0  #swap wins
dontswap=0 #dontswap wins
j=0
while(j<10):
    x=random.randint(0,2)
    doors[x]="bmw"
    for i in range(0,3):
        if i==x:
            continue
        else:
            doors[i]="goat"
            goatdoor.append(i)
    choice=int(input("enter choice:"))
    door_open=random.choice(goatdoor) #only goat
    while door_open==choice:
        door_open=random.choice(goatdoor) #not equal to choice
    ch=input("swap????:y/n:")
    if ch=='y':
        if doors[choice]=="goat":
            print("wins")
            swap=swap+1
        else:
            print("lost")
    else:
        if doors[choice]=="goat":
            print("lost")
        else:
            print("wins")
            dontswap=dontswap+1
    j=j+1
print("swap:",swap)
print("dontswap",dontswap)
```

```
enter choice: 2
swap????:y/n: y
```

```
wins
```

```
enter choice: 2
swap????:y/n: n
```

```
lost
```

```
enter choice: 2
swap????:y/n: y

wins

enter choice: 2
swap????:y/n: n

lost

enter choice: 2
swap????:y/n: n

wins
```

0.12 rock paper scissors

[5]:
```python
def rock_paper_scissors(n1,n2,bit1,bi2):
    p1choice=int(n1[bit1])%3
    p2choice=int(n2[bit2])%3
    if p1[p1choice]==p2[p2choice]:
        print("draw")
    elif p1[p1choice]=='rock' and p2[p2choice]=='scissors':
        print("player1 wins")
    elif p1[p1choice]=='rock' and p2[p2choice]=='paper':
        print("player2 wins")
    elif p1[p1choice]=='paper' and p2[p2choice]=='scissors':
        print("player2 wins")
    elif p1[p1choice]=='paper' and p2[p2choice]=='rock':
        print("player1 wins")
    elif p1[p1choice]=='scissors' and p2[p2choice]=='paper':
        print("player1 wins")
    else:
        print("player2 wins")

p1={0:'rock',1:'paper',2:'scissors'}
p2={1:'paper',1:'rock',2:'scissors'}
while(1):
    n1=input("player 1 choice:")
    n2=input("palyer 2 choice:")
    bit1=int(input("player1,enter secret bit pos:"))
    bit2=int(input("player2,enter secret bit pos:"))
    rock_paper_scissors(n1,n2,bit1,bit2)
    ch=input("cont..? y/n:")
    if ch=='n':
        break
```

```
player 1 choice: 1234567
palyer 2 choice: 729181
```

```
player1,enter secret bit pos: 3
player2,enter secret bit pos: 4

player2 wins

cont..? y/n: y
player 1 choice: 2314562313564232
palyer 2 choice: 47235733521
player1,enter secret bit pos: 9
player2,enter secret bit pos: 9

player1 wins

cont..? y/n: n
```

0.13 searching and sorting

[7]:
```
## binary search
#bubble sort
def bubble(a):
    n=len(a)
    for i in range(n):
        for j in range(0,n-i-1):
            if a[j] > a[j+1]:
                temp=a[j]
                a[j]=a[j+1]
                a[j+1]=temp
a=[5,1,4,2,8]
bubble(a)
for i in a:
    print(i)
```

```
1
2
4
5
8
```

[19]:
```
#linear search
def linear_search(n,x):
    a=[]
    for i in range(1,n+1):
        a.append(i)
    c=0
    flag=0
    for i in a:
        c+=1
        if i==x:
            print(i,"found at pos:"+str(i))
```

```
            flag=1
            break;
    if flag==0:
        print(" not found")
    print("no of iterations:",c)
linear_search(10,4)
linear_search(10,9)
linear_search(10,21)
```

```
4 found at pos:4
no of iterations: 4
9 found at pos:9
no of iterations: 9
 not found
no of iterations: 10
```

[1]:
```
def binary_search(a,x):
    low=0
    high=len(a)-1
    flag=0
    c=0
    while(low<=high and flag==0):
        c+=1
        mid=(low+high) //2
        if x==a[mid]:
            flag=1
            print("found at:"+str(mid))
            print("no of iterations:"+str(c))
            return
        else:
            if x<a[mid]:
                high=mid-1
            else:
                low=mid+1
    print("not found")
a=[]
for i in range(1,10000):
    a.append(i)
binary_search(a,234)
```

```
found at:233
no of iterations:7
```

0.14 Caesar Ciphers

[7]:
```
import string
a="hello world"
print(a)
print(a[0])
print(a.upper)
print(a[2:5])
print(len(a))
a.replace('h','s')
```

```
hello world
h
<built-in method upper of str object at 0x7fdc9014bd30>
llo
11
```

[7]: 'sello world'

[8]:
```
print(string.ascii_letters)
```

```
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
```

[9]:
```
import string
dict={}
for i in range(len(string.ascii_letters)):
    dict[string.ascii_letters[i]]=string.ascii_letters[i-1]
print(dict)
```

```
{'a': 'Z', 'b': 'a', 'c': 'b', 'd': 'c', 'e': 'd', 'f': 'e', 'g': 'f', 'h': 'g',
'i': 'h', 'j': 'i', 'k': 'j', 'l': 'k', 'm': 'l', 'n': 'm', 'o': 'n', 'p': 'o',
'q': 'p', 'r': 'q', 's': 'r', 't': 's', 'u': 't', 'v': 'u', 'w': 'v', 'x': 'w',
'y': 'x', 'z': 'y', 'A': 'z', 'B': 'A', 'C': 'B', 'D': 'C', 'E': 'D', 'F': 'E',
'G': 'F', 'H': 'G', 'I': 'H', 'J': 'I', 'K': 'J', 'L': 'K', 'M': 'L', 'N': 'M',
'O': 'N', 'P': 'O', 'Q': 'P', 'R': 'Q', 'S': 'R', 'T': 'S', 'U': 'T', 'V': 'U',
'W': 'V', 'X': 'W', 'Y': 'X', 'Z': 'Y'}
```

[1]:
```
import string
dict={}
data=""
file=open("helo1.txt","w")
for i in range(len(string.ascii_letters)):
    dict[string.ascii_letters[i]]=string.ascii_letters[i-1]
print(dict)
with open("helo.txt") as f:
    while True:
        c=f.read(1)
        if not c:
```

```
            print("end of file")
            break
        if c in dict:
            data=dict[c]
        else:
            data=c
            file.write(data)
        print(data)
file.close()
```

```
{'a': 'Z', 'b': 'a', 'c': 'b', 'd': 'c', 'e': 'd', 'f': 'e', 'g': 'f', 'h': 'g',
'i': 'h', 'j': 'i', 'k': 'j', 'l': 'k', 'm': 'l', 'n': 'm', 'o': 'n', 'p': 'o',
'q': 'p', 'r': 'q', 's': 'r', 't': 's', 'u': 't', 'v': 'u', 'w': 'v', 'x': 'w',
'y': 'x', 'z': 'y', 'A': 'z', 'B': 'A', 'C': 'B', 'D': 'C', 'E': 'D', 'F': 'E',
'G': 'F', 'H': 'G', 'I': 'H', 'J': 'I', 'K': 'J', 'L': 'K', 'M': 'L', 'N': 'M',
'O': 'N', 'P': 'O', 'Q': 'P', 'R': 'Q', 'S': 'R', 'T': 'S', 'U': 'T', 'V': 'U',
'W': 'V', 'X': 'W', 'Y': 'X', 'Z': 'Y'}
v
g
d
m
n
o
d
m
d
c
a
x
Z
s
d
w
s
d
c
h
s
n
q
g
t
l
Z
m
q
d
```

Z
c
Z
a
k
d
b
n
m
s
d
m
s
h
r
o
q
d
r
d
m
s
d
c
s
n
s
g
d
t
r
d
q
s
g
h
r
n
e
s
d
m
b
n
m
r
h
r

s
r
n
e
s
g
d
e
h
k
d
r
o
k
Z
h
m
s
d
w
s
u
h
r
h
a
k
d
s
n
s
g
d
t
r
d
q
c
d
o
d
m
c
h
m
f
n
m

s
g
d
Z
o
o
k
h
b
Z
s
h
n
m
b
n
m
s
q
n
k
b
n
c
d
r
l
Z
x
a
d
q
d
m
c
d
q
d
c
d
h
s
g
d
q
Z
r
k

h
s
d
q
Z
k
h
m
r
s
q
t
b
s
h
n
m
r
Z
b
s
d
c
t
o
n
m
a
x
s
g
d
d
c
h
s
n
q
n
q
Z
r
u
h
r
h
a
k

d
d
r
b
Z
o
d
b
g
Z
q
Z
b
s
d
q
r
s
g
Z
s
b
Z
m
a
d
d
c
h
s
d
c
Z
r
o
k
Z
h
m
s
d
w
s
s
g
n
t
f

g
s
g
d
q
d
l
Z
x
a
d
o
k
Z
h
m
s
d
w
s
h
m
Z
s
d
w
s
e
h
k
d
b
n
m
s
q
n
k
b
g
Z
q
Z
b
s
d
q
r

v
h
s
g
h
m
s
g
d
e
h
k
d
d
r
o
d
b
h
Z
k
k
x
s
g
d
d
m
c
n
e
e
h
k
d
b
g
Z
q
Z
b
s
d
q
b
Z
m
q

```
d
m
c
d
q
s
g
d
o
k
Z
h
m
s
d
w
s
t
m
r
d
d
m
a
x
Z
o
Z
q
s
h
b
t
k
Z
q
l
d
s
g
n
c
end of file
```

0.15 tic tac toe

[6]:
```
import numpy as np
board=np.array([['-','-','-'],['-','-','-'],['-','-','-']])
p1='X'
p2='O'
def place(sym):
    print(np.matrix(board))
    while(1):
        row=int(input("enter row(1/2/3):"))
        col=int(input("enter col(1/2/3):"))
        if row >0 and row<4 and col>0 and col<4 and board[row-1][col-1]=='-':
            break
        else:
            print("invalid or else occupied,enter again")
    board[row-1][col-1]=sym

def won(sym):
    return check_rows(sym) or check_cols(sym) or check_diagonals(sym)

def check_rows(sym):
    for r in range(3):
        count=0
        for c in range(3):
            if board[r][c]==sym:
                count+=1
        if count==3:
            print(sym," won")
            return True
    return False

def check_cols(sym):
    for c in range(3):
        count=0
        for r in range(3):
            if board[r][c]==sym:
                count+=1
        if count==3:
            print(sym," won")
            return True
    return False

def check_diagonals(sym):
    if board[0][2] == board[1][1]==board[2][0]==sym:
        print(sym," won diagonal aligned")
        return True
    count=0
    for r in range(3):
```

```
        for c in range(3):
            if r==c:
                if board[r][c]==sym:
                    count+=1
    if count==3:
        print(sym," won diagonal loop")
        return True
    return False

def play():
    for turn in range(9):
        if turn%2==0:
            print('X ur turn')
            place(p1)
            if won(p1):
                break
        else:
            print('O ur turn')
            place(p2)
            if won(p2):
                break
    if not(won(p1)) and not(won(p2)):
        print("draw")
play()
```

```
X ur turn
[['-' '-' '-']
 ['-' '-' '-']
 ['-' '-' '-']]

enter row(1/2/3): 1
enter col(1/2/3): 1

O ur turn
[['X' '-' '-']
 ['-' '-' '-']
 ['-' '-' '-']]

enter row(1/2/3): 2
enter col(1/2/3): 2

X ur turn
[['X' '-' '-']
 ['-' 'O' '-']
 ['-' '-' '-']]

enter row(1/2/3): 1
enter col(1/2/3): 3

O ur turn
```

```
[['X' '-' 'X']
 ['-' 'O' '-']
 ['-' '-' '-']]

enter row(1/2/3): 2
enter col(1/2/3): 3

X ur turn
[['X' '-' 'X']
 ['-' 'O' 'O']
 ['-' '-' '-']]

enter row(1/2/3): 1
enter col(1/2/3): 2

X  won
X  won
```

0.16 Recursion

[1]:
```python
def factorial(n):
    product=1
    for i in range(1,n+1):
        product=product*i
    return product
n=int(input("Enter positive num:"))
if n<0:
    print("Factorial not defined for negative num")
else:
    f=factorial(n)
    print("factorial of ",n,"is ",f)
```

```
Enter positive num: 3

factorial of  3 is  6
```

[2]:
```python
def fact(n):
    # fact(n)=n*fact(n-1)
    # fact(0)=1
    #base case
    if n==0:
        return 1
    else:
        return n*fact(n-1)
n=int(input("Enter positive num:"))
if n<0:
    print("Factorial not defined for negative num")
else:
    f=fact(n)
    print("factorial of ",n,"is ",f)
```

```
Enter positive num: 5

factorial of  5 is  120
```

[4]:
```python
#binary search using recursion
def binary_search(l,x,start,end):
    #base case 1 element
    if start==end:
        if l[start]==x:
            return start
        else:
            return -1
    else:
        mid=int((start+end)/2)
        if l[mid]==x:
            return mid
        elif l[mid]>x:
            return binary_search(l,x,start,mid-1)
        elif l[mid]<x:
            return binary_search(l,x,mid+1,end)
l=[2,23,46,78,80,94]
x=int(input("Enter search key:"))
index=binary_search(l,x,0,len(l)-1)
if index==-1:
    print("not found")
else:
    print(x," is found at ",index+1)
```

```
Enter search key: 46

46  is found at  3
```

[6]:
```python
#fibonacci sequence
# 0,1,1,2,3,5,8,13,..
def fib(n):
    if n<2:
        return n
    else:
        return fib(n-1)+fib(n-2)
n=int(input("Enter positive num:"))
if n<0:
    print("Factorial not defined for negative num")
else:
    f=fib(n)
    print(f," is num at ",n,"th position")
```

```
Enter positive num: 4

3  is num at  4 th position
```

0.17 Snake and Ladders

[14]:
```
from PIL import Image
import random
end=100
def show_board():
    img=Image.open('snl.png')
    img.show()

def check_ladder(points):
    if points==8:
        print('Ladder')
        return 26
    elif points==21:
        print('Ladder')
        return 82
    elif points==43:
        print('Ladder')
        return 77
    elif points==50:
        print('Ladder')
        return 91
    elif points==54:
        print('Ladder')
        return 93
    elif points==62:
        print('Ladder')
        return 96
    elif points==66:
        print('Ladder')
        return 87
    elif points==80:
        print('Ladder')
        return 100
    else:
        return points

def check_snake(points):
    if points==44:
        print("Snake")
        return 22
    elif points==46:
        print("Snake")
        return 5
    elif points==48:
        print("Snake")
        return 9
    elif points==52:
```

```
        print("Snake")
        return 11
    elif points==55:
        print("Snake")
        return 7
    elif points==59:
        print("Snake")
        return 17
    elif points==64:
        print("Snake")
        return 36
    elif points==69:
        print("Snake")
        return 33
    elif points==73:
        print("Snake")
        return 1
    elif points==83:
        print("Snake")
        return 19
    elif points==92:
        print("Snake")
        return 51
    elif points==95:
        print("Snake")
        return 24
    elif points==98:
        print("Snake")
        return 28
    else:
        return points
def reached_end(points):
    if points==end:
        return True
    else:
        return False

def play():
    p1=input('player1 name:')
    p2=input('player2 name:')
    #initial points
    pp1=0
    pp2=0
    turn=0
    while(1):
        if turn%2==0:
            print(p1," ur turn")
```

```
            c=int(input("1 to cont..,0 to quit:"))
            if c==0:
                print(p1," score is:",pp1)
                print(p2," score is:",pp2)
                break
            #generate dice
            dice=random.randint(1,6)
            print("Dice is:",dice)
            pp1+=dice
            pp1=check_ladder(pp1)
            pp1=check_snake(pp1)
            if pp1>end:
                pp1=end
            print(p1," score is :",pp1)
            if reached_end(pp1):
                print(p1," won")
                break
        else:
            print(p2," ur turn")
            c=int(input("1 to cont..,0 to quit:"))
            if c==0:
                print(p1," score is:",pp1)
                print(p2," score is:",pp2)
                break
            #generate dice
            dice=random.randint(1,6)
            print("Dice is:",dice)
            pp2+=dice
            pp2=check_ladder(pp2)
            pp2=check_snake(pp2)
            if pp2>end:
                pp2=end
            print(p2," score is :",pp2)
            if reached_end(pp2):
                print(p2," won")
                break
        turn+=1

show_board()
play()
```

```
player1 name: sneha
player2 name: suresh

sneha  ur turn

1 to cont..,0 to quit: 1

Dice is: 4
```

```
sneha  score is : 4
suresh  ur turn

1 to cont..,0 to quit: 1

Dice is: 4
suresh  score is : 4
sneha  ur turn

1 to cont..,0 to quit: 1

Dice is: 4
Ladder
sneha  score is : 26
suresh  ur turn

1 to cont..,0 to quit: 1

Dice is: 6
suresh  score is : 10
sneha  ur turn

1 to cont..,0 to quit: 1

Dice is: 6
sneha  score is : 32
suresh  ur turn

1 to cont..,0 to quit: 1

Dice is: 3
suresh  score is : 13
sneha  ur turn

1 to cont..,0 to quit: 1

Dice is: 1
sneha  score is : 33
suresh  ur turn

1 to cont..,0 to quit: 1

Dice is: 6
suresh  score is : 19
sneha  ur turn

1 to cont..,0 to quit: 1

Dice is: 4
sneha  score is : 37
suresh  ur turn

1 to cont..,0 to quit: 1

Dice is: 2
Ladder
```

```
suresh  score is : 82
sneha  ur turn

1 to cont..,0 to quit: 1

Dice is: 3
sneha  score is : 40
suresh  ur turn

1 to cont..,0 to quit: 1

Dice is: 4
suresh  score is : 86
sneha  ur turn

1 to cont..,0 to quit: 1

Dice is: 1
sneha  score is : 41
suresh  ur turn

1 to cont..,0 to quit: 1

Dice is: 5
suresh  score is : 91
sneha  ur turn

1 to cont..,0 to quit: 1

Dice is: 4
sneha  score is : 45
suresh  ur turn

1 to cont..,0 to quit: 1

Dice is: 5
suresh  score is : 96
sneha  ur turn

1 to cont..,0 to quit: 0

sneha  score is: 45
suresh  score is: 96
```

0.18 Spiral Traversing

[8]:
```
def spiral(m,n,a):
    k=l=0
    #first row
    while k<m and l<n:
        for i in range(l,n):
            print(a[k][i],end=" ")
        k+=1
        #last col
```

```
        for i in range(k,m):
            print(a[i][n-1],end=" ")
        n-=1
        if k<m:
            #last row reverse remain
            for i in range(n-1,l-1,-1):
                print(a[m-1][i],end=" ")
        m-=1
        if l<n:
            #first col reverse remain
            for i in range(m-1,k-1,-1):
                print(a[i][l],end=" ")
            l+=1
a=[]
count=1
for i in range(4):
    l=[]
    for j in range(4):
        l.append(count)
        count+=1
    a.append(l)
spiral(4,4,a)
```

```
1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10
```

0.19 turtle

[1]:
```
from tkinter import *
import turtle
t=turtle.Turtle()
for i in range(5):
    t.forward(50)
    t.right(90)
turtle.done()
```

[1]:
```
from tkinter import *
import turtle
t=turtle.Turtle()
for i in range(5):
    t.forward(50)
    t.right(144)
turtle.done()
```

[1]:
```
from tkinter import *
import turtle
turtle.bgcolor("black")
seurat=turtle.Turtle()
```

```
width=5
height=7
dot_distance=25
seurat.setpos(-250,250)

def spiral(m,n):
    k=l=0
    f=0
    seurat.color("white")
    #first row
    while k<m and l<n:
        if f==1:
            seurat.right(90)
        for i in range(l,n):
            #print(a[k][i],end=" ")
             seurat.forward(dot_distance)
        k+=1
        f=1

        seurat.right(90)
        #last col
        for i in range(k,m):
            #print(a[i][n-1],end=" ")
            seurat.forward(dot_distance)
        n-=1

        seurat.right(90)
        if k<m:
            #last row reverse remain
            for i in range(n-1,l-1,-1):
                #print(a[m-1][i],end=" ")
                seurat.forward(dot_distance)
        m-=1

        seurat.right(90)
        if l<n:
            #first col reverse remain
            for i in range(m-1,k-1,-1):
                #print(a[i][l],end=" ")
                seurat.forward(dot_distance)
            l+=1


spiral(20,20)
```

[1]:
```
from tkinter import *
import turtle
```

```
from random import randint
turtle.bgcolor("black")
seurat=turtle.Turtle()
width=5
height=7
dot_distance=25

seurat.penup()
list_color=["white","yellow","brown","red","blue","green","orange","pink","violet","grey","cya

seurat.setpos(-250,250)


def spiral(m,n):
    k=l=0
    f=0
    seurat.color("white")

    # k-start row index
    # m-end row index
    # l- start col index
    # n-end col index
    # i-iterator

    col=randint(0,11)
    seurat.color(list_color[col])

    #first row
    while k<m and l<n:
        if f==1:
            seurat.right(90)
        for i in range(l,n):
            #print(a[k][i],end=" ")
            seurat.dot()
            seurat.forward(dot_distance)
        k+=1
        f=1

        seurat.right(90)
        col=randint(0,11)
        seurat.color(list_color[col])
        #last col
        for i in range(k,m):
            #print(a[i][n-1],end=" ")
            seurat.dot()
            seurat.forward(dot_distance)
        n-=1
```

```
        seurat.right(90)
        col=randint(0,11)
        seurat.color(list_color[col])
        if k<m:
            #last row reverse remain
            for i in range(n-1,l-1,-1):
                #print(a[m-1][i],end=" ")
                seurat.dot()
                seurat.forward(dot_distance)
        m-=1

        seurat.right(90)
        col=randint(0,11)
        seurat.color(list_color[col])
        if l<n:
            #first col reverse remain
            for i in range(m-1,k-1,-1):
                #print(a[i][l],end=" ")
                seurat.dot()
                seurat.forward(dot_distance)
            l+=1


spiral(20,20)
```

0.20 GPS-Global Positioning System

[5]:
```
from gmplot import gmplot
import csv
gmap=gmplot.GoogleMapPlotter(28.686086,77.325074,12)
gmap.coloricon="http://www.googlemapsmarkers.com/v1/%s/"
with open("route.csv","r") as f:
    reader=csv.reader(f)
    k=0
    for row in reader:
        lat=float(row[0])
        long=float(row[1])
        print(lat)
        print(long)
        if k==0:
            gmap.marker(lat,long,"yellow")
            k=1
        else:
            gmap.marker(lat,long,"blue")
gmap.marker(lat,long,"red")
gmap.draw("mymap.html")
```

```
#open myhtml to get result of map
```

```
28.686086
77.325074
28.685742
77.325305
28.685295
77.325608
28.684968
77.325847
28.684542
77.326147
28.684288
77.326316
28.68398
77.326515
28.683756
77.326094
28.683573
77.32578
```

0.21 Tuples

[6]:
```
ice=("vanilla","chocolate","butterscotch","strawberry")
ice
```

[6]:
```
('vanilla', 'chocolate', 'butterscotch', 'strawberry')
```

[7]:
```
for flavour in ice:
    print(flavour)
```

```
vanilla
chocolate
butterscotch
strawberry
```

[8]:
```
ice[1]
```

[8]:
```
'chocolate'
```

[9]:
```
ice[14]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 ice[14]
```

```
IndexError: tuple index out of range
```

[10]:
```
ice[2]="blackcurrent"
ice
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[10], line 1
----> 1 ice[2]="blackcurrent"
      2 ice

TypeError: 'tuple' object does not support item assignment
```

[11]:
```
ice[4]="blackcurrent"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[11], line 1
----> 1 ice[4]="blackcurrent"

TypeError: 'tuple' object does not support item assignment
```

[12]:
```
del ice[1]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[12], line 1
----> 1 del ice[1]

TypeError: 'tuple' object doesn't support item deletion
```

[13]:
```
len(ice)
```

[13]: 4

[14]:
```
del ice
ice
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[14], line 2
      1 del ice
----> 2 ice
```

```
NameError: name 'ice' is not defined
```

[17]:
```
t=(1,2,3,4)
len(t)
print(t.count(2))
t=(1,2,3,4,3,2,2,1,2)
t.count(2)
```

```
1
```

[17]: 4

[18]:
```
t.index(2)
```

[18]: 1

[20]:
```
rainbow=("violet","indigo","blue","green","orange","yellow","red","pink")
rainbow = rainbow + (4,)
rainbow
```

[20]: ('violet', 'indigo', 'blue', 'green', 'orange', 'yellow', 'red', 'pink', 4)

0.22 Lottery Simualtion

[23]:
```
import random
bet=int(input("your bet from 1 to 10:"))
lucky_draw=random.randint(1,10)
print(lucky_draw)
account=0
if bet==lucky_draw:
    account=account+900-100
else:
    account=account-100
print(account)
```

```
your bet from 1 to 10: 4

1
-100
```

[25]:
```
import random
account=0
for i in range(7):
    bet=random.randint(1,10)
    lucky_draw=random.randint(1,10)
    print("Bet:",bet)
```

```
    print("Lucky draw:",lucky_draw)
    if bet==lucky_draw:
        account=account+900-100
    else:
        account=account-100
    print("Amount in ur account:",account)
```

```
Bet: 5
Lucky draw: 5
Amount in ur account: 800
Bet: 9
Lucky draw: 10
Amount in ur account: 700
Bet: 6
Lucky draw: 6
Amount in ur account: 1500
Bet: 8
Lucky draw: 6
Amount in ur account: 1400
Bet: 1
Lucky draw: 8
Amount in ur account: 1300
Bet: 7
Lucky draw: 1
Amount in ur account: 1200
Bet: 5
Lucky draw: 8
Amount in ur account: 1100
```

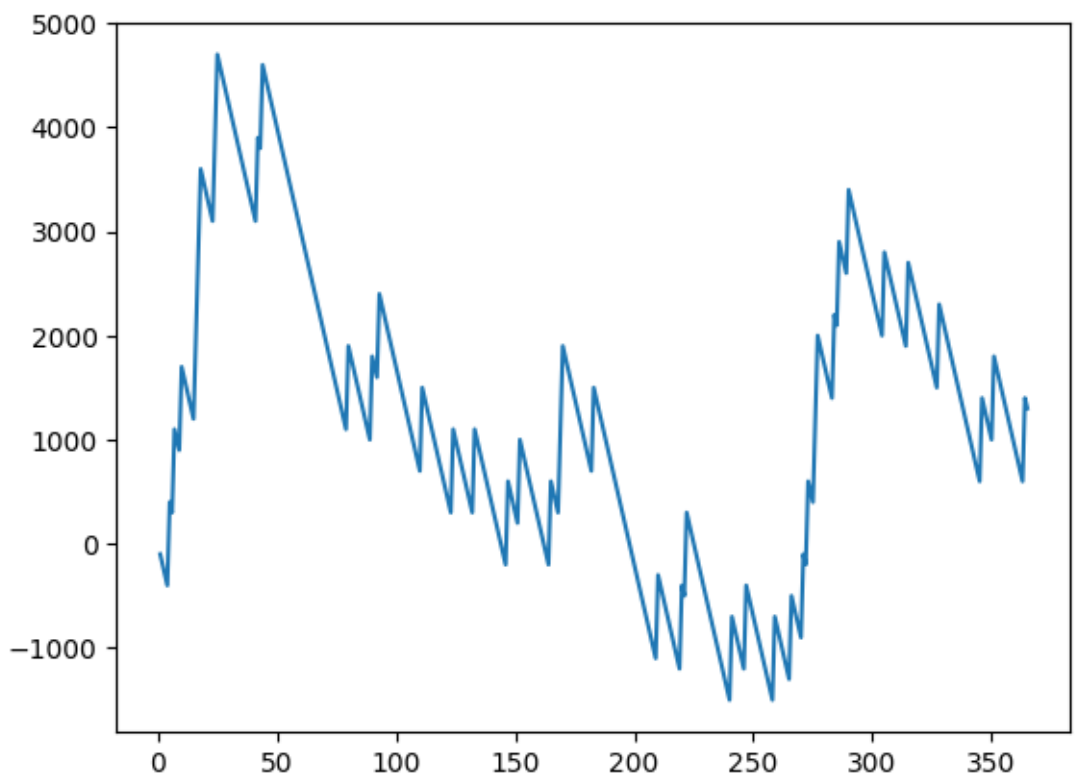
[28]:
```
import random
account=0
for i in range(30):
    bet=random.randint(1,10)
    lucky_draw=random.randint(1,10)
    # print("Bet:",bet)
    # print("Lucky draw:",lucky_draw)
    if bet==lucky_draw:
        account=account+900-100
    else:
        account=account-100
print("Amount in ur account:",account)
```

```
Amount in ur account: 1500
```

[32]:
```
import random
import matplotlib.pyplot as plt
account=0
```

```
x=[]
y=[]
for i in range(365):
    x.append(i+1)
    bet=random.randint(1,10)
    lucky_draw=random.randint(1,10)
    # print("Bet:",bet)
    # print("Lucky draw:",lucky_draw)
    if bet==lucky_draw:
        account=account+900-100
    else:
        account=account-100
    y.append(account)
print("Amount in ur account:",account)
plt.plot(x,y)
plt.show()
```

```
Amount in ur account: 1300
```



0.23 Image Processing

[1]:
```
#flipping image
from PIL import Image
#open
img=Image.open("tobe.png")
#transposing
transposed_img=img.transpose(Image.FLIP_LEFT_RIGHT)
#save for human understandable
transposed_img.save('corrected.png') #directory
print("Done flipping")
```

```
Done flipping
```

[5]:
```
 #image enhancement CLAHE
import cv2
#read image
img=cv2.imread('tobe.jpeg')
#preparation for clahe
clahe=cv2.createCLAHE()
#convert to gray scale
gray_img=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
#apply enhancement
enh_img=clahe.apply(gray_img)
#human undearstandable
cv2.imwrite('enhanced.png',enh_img)
print("Done enhancing")
```

```
Done enhancing
```

0.24 Anagram =combination of same letters

[33]:
```
x=[2,1,3,7,4]
print(sorted(x))
print(sorted(x,reverse=True))
```

```
[1, 2, 3, 4, 7]
[7, 4, 3, 2, 1]
```

[34]:
```
x=['q','w','e','r','y']
print(sorted(x))
print(sorted(x,reverse=True))
```

```
['e', 'q', 'r', 'w', 'y']
['y', 'w', 'r', 'q', 'e']
```

[35]:
```
x=['sneha','ara','erw']
print(sorted(x))
```

```
print(sorted(x,reverse=True))
```

```
['ara', 'erw', 'sneha']
['sneha', 'erw', 'ara']
```

[36]:
```
x={'q':1,'w':2,'e':3,'r':0}
#based on keys
print(sorted(x))
print(sorted(x,reverse=True))
```

```
['e', 'q', 'r', 'w']
['w', 'r', 'q', 'e']
```

[37]:
```
x=["cccccc","bb","aaa","dddddddd"]
print(sorted(x,key=len))
```

```
['bb', 'aaa', 'cccccc', 'dddddddd']
```

[40]:
```
str1=input("enter  string1:")
str2=input("enter  string2:")
if sorted(str1) == sorted(str2):
    print("Anagram")
else:
    print("Not a Anagram")
```

```
enter  string1: sneha
enter  string2: snehh
```

```
Not a Anagram
```

[4]:
```
l=input()
k=int(input())
l=l.split()
a=[]
for i in range(0,len(l)):
  a.append(k*int(l[i]))
a.sort()
print(a[-1])
```

```
 4 7 2 8
 3
```

```
24
```

1 Facebook analysis

[7]:
```
import pandas as pd
import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer
nltk.downloader.download('vader_lexicon')
file='Facebook_data.xlsx'
xl=pd.ExcelFile(file) #Read From Excel
dfs = xl.parse(xl.sheet_names[0]) #Pasing the sl sheet to data frame
dfs=list(dfs['25/08/2018 19:49:00 + UTC']) #removes blank rows from the data␣
 ↪frame
print(dfs)
sid = SentimentIntensityAnalyzer()
str1='UTC'
for dat in dfs:
    a = dat.find(str1)
    if(a == -1):
        ss=sid.polarity_scores(dat)
        print(dat)
        for k in ss:
            print(k,ss[k])
```

```
[nltk_data] Downloading package vader_lexicon to
[nltk_data]     /home/rgukt/nltk_data…
[nltk_data]   Package vader_lexicon is already up-to-date!
```

```
['I have taken cs50 long back Actually I am presently developing an app named
Scheduler a simple todo list easily accessible to every one i source code and
apk is on github https://github.com/dnmanveet/Scheduler I need some suggestion
and reviews for my app i just wanna release it in play store so please to
suggest me any changes in made even is silly it will be considered please to
review my app', nan, '12/08/2018 21:07:00 + UTC', 'Actually i have taken up cs50
long back i think it was last year jun i have completed aronud 6 psets in my ide
and submitted them, and got the result too. but I dont see in my github can any
one tell me how do I upload it on my github account', nan, '31/01/2018 17:31:00+
UTC', 'Can anyone help me telling the name of course to learn php language plz',
nan, '22/12/2017 16:43:00+ UTC', 'Happy birtrhday ra', nan, '22/12/2017
16:43:00+ UTC', 'In pset 6 after submitting the pset will we get the percentage
scored??\ni have submitted my pset by i didnt get any percentage my result is in
the linkhttps://harvard.az1.qualtrics.com/CP/Report.php?SID=SV_8qOQ32gOCmg8ijz&R
=R_1IlYvR92WUbX5Iy\nplz help me by by saying this!!!!!!', nan, '12/10/2017
23:18:00 + UTC', 'ENJOY CODING IN PYTHON!!!very interesting thing very tough
very sweet output!!!! to taste it after a long struggle understanding !!!!!']
I have taken cs50 long back Actually I am presently developing an app named
Scheduler a simple todo list easily accessible to every one i source code and
apk is on github https://github.com/dnmanveet/Scheduler I need some suggestion
and reviews for my app i just wanna release it in play store so please to
suggest me any changes in made even is silly it will be considered please to
```

```
review my app
neg 0.0
neu 0.844
pos 0.156
compound 0.8437
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[7], line 13
     11 str1='UTC'
     12 for dat in dfs:
---> 13     a = dat.find(str1)
     14     if(a == -1):
     15         ss=sid.polarity_scores(dat)

AttributeError: 'float' object has no attribute 'find'
```

2 Stylometry

[12]:
```
papers={ 'Madison': [10,14,37,38,39,40,41,42,43,44,45,46,47,48],
        'Hamilton':[1,6,7,8,9,11,12,13,15,16,17,21,22.
 ↪23,24,25,26,27,28,29,30,31,
                    ␣
 ↪32,33,34,35,36,59,60,61,65,66,67,68,69,70,71,72,73,74,75,76,77,
                     78,79,80,81,82,83,84,85],
        'Jay':[2,3,4,5],
        'Shared':[18,19,20],
        'Disputed':[49,50,51,52,53,54,55,56,57,58,62,63]
       }
def read_files(filename):
    strings=[]
    for file in filename:
        with open(f'federalist_{filename}.txt') as f:
            strings.append(f.read())
    return ('\n'.join(strings))

federalist_by_author={}
for author,files in papers.items():
    federalist_by_author[author]=read_files(files)
for author in papers:
    print(fedaralist_by_author[author][:100])
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[12], line 18
```

```
     16 federalist_by_author={}
     17 for author,files in papers.items():
---> 18     federalist_by_author[author]=read_files(files)
     19 for author in papers:
     20     print(fedaralist_by_author[author][:100])

Cell In[12], line 12, in read_files(filename)
     10 strings=[]
     11 for file in filename:
---> 12     with open(f'federalist_{filename}.txt') as f:
     13         strings.append(f.read())
     14 return ('\n'.join(strings))

File ~/.local/lib/python3.10/site-packages/IPython/core/interactiveshell.py:324␣
 ↪in _modified_open(file, *args, **kwargs)
    317 if file in {0, 1, 2}:
    318     raise ValueError(
    319         f"IPython won't let you open fd={file} by default "
    320         "as it is likely to crash IPython. If you know what you are␣
 ↪doing, "
    321         "you can use builtins' open."
    322     )
--> 324 return io_open(file, *args, **kwargs)

FileNotFoundError: [Errno 2] No such file or directory: 'federalist_[10, 14, 37␣
 ↪38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48].txt'
```

[13]:
```
import nltk
papers={ 'Madison': [10,14,37,38,39,40,41,42,43,44,45,46,47,48],
        'Hamilton':[1,6,7,8,9,11,12,13,15,16,17,21,22.
 ↪23,24,25,26,27,28,29,30,31,
                    ␣
 ↪32,33,34,35,36,59,60,61,65,66,67,68,69,70,71,72,73,74,75,76,77,
                     78,79,80,81,82,83,84,85],
        'Jay':[2,3,4,5],
        'Shared':[18,19,20],
        'Disputed':[49,50,51,52,53,54,55,56,57,58,62,63]
       }
def read_files(filename):
    strings=[]
    for file in filename:
        with open(f'federalist_{filename}.txt') as f:
            strings.append(f.read())
    return ('\n'.join(strings))

federalist_by_author={}
```

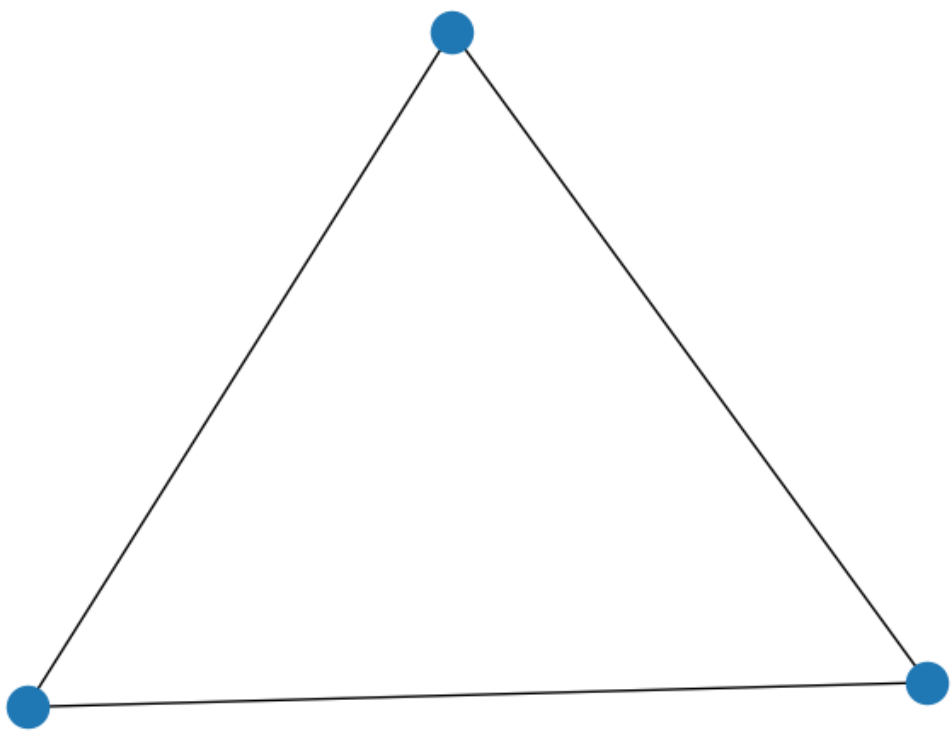
```python
for author,files in papers.items():
    federalist_by_author[author]=read_files(files)

authors=("Hamilton","Madison","Disputed","Jay","Shared")
author_tokens={}
length_distribution={}
for author in authors:
    tokens=nltk.word_tokenize(fedaralist_by_author[author])
    author_tokens[author]=([token for token in tokens if any(c.isalpha() for c␣
 ↪in token)])
    token_lengths=[len(token)for token in author_token[author]]
    length_distributions[authors]=nlyk.FreDist(token_lengths)
    length_distributions[author].plot(15,title=author)
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[13], line 20
     17 federalist_by_author={}
     19 for author,files in papers.items():
---> 20     federalist_by_author[author]=read_files(files)
     22 authors=("Hamilton","Madison","Disputed","Jay","Shared")
     23 author_tokens={}

Cell In[13], line 13, in read_files(filename)
     11 strings=[]
     12 for file in filename:
---> 13     with open(f'federalist_{filename}.txt') as f:
     14         strings.append(f.read())
     15 return ('\n'.join(strings))

File ~/.local/lib/python3.10/site-packages/IPython/core/interactiveshell.py:324 ␣
 ↪in _modified_open(file, *args, **kwargs)
    317 if file in {0, 1, 2}:
    318     raise ValueError(
    319         f"IPython won't let you open fd={file} by default "
    320         "as it is likely to crash IPython. If you know what you are␣
 ↪doing, "
    321         "you can use builtins' open."
    322     )
--> 324 return io_open(file, *args, **kwargs)

FileNotFoundError: [Errno 2] No such file or directory: 'federalist_[10, 14, 37 ␣
 ↪38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48].txt'
```

3 Network analysis

[21]:
```
import networkx as nx
G=nx.Graph()
G.add_node(1)
G.add_node(2)
G.add_node(3)
G.add_edge(1,2)
G.add_edge(2,3)
G.add_edge(1,3)
print(G.nodes())
print(G.edges())
nx.draw(G)
```

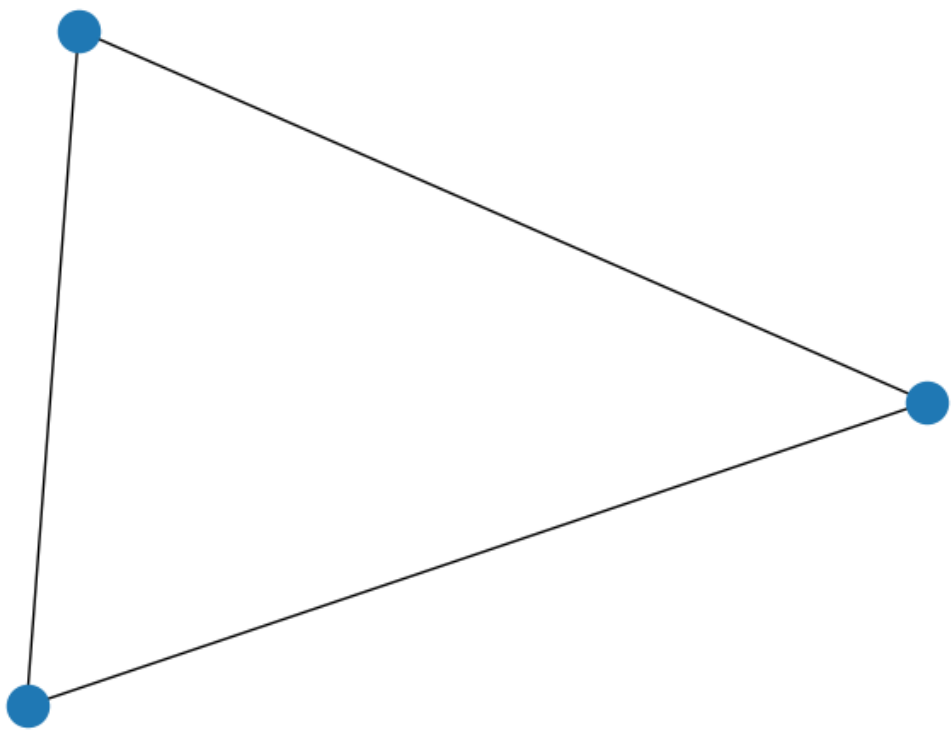
```
[1, 2, 3]
[(1, 2), (1, 3), (2, 3)]
```



[20]:
```
import networkx as nx
import matplotlib .pyplot as plt
G=nx.Graph()
G.add_node(1)
```

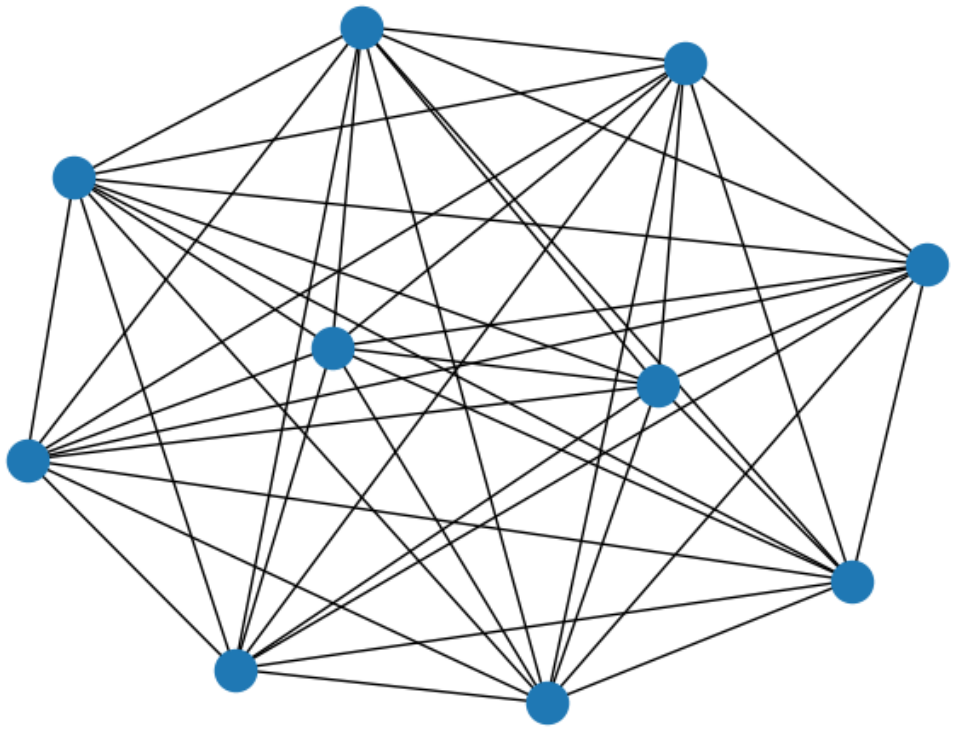
```
G.add_node(2)
G.add_node(3)
G.add_edge(1,2)
G.add_edge(2,3)
G.add_edge(1,3)
print(G.nodes())
print(G.edges())
nx.draw(G)
plt.show()
```

```
[1, 2, 3]
[(1, 2), (1, 3), (2, 3)]
```

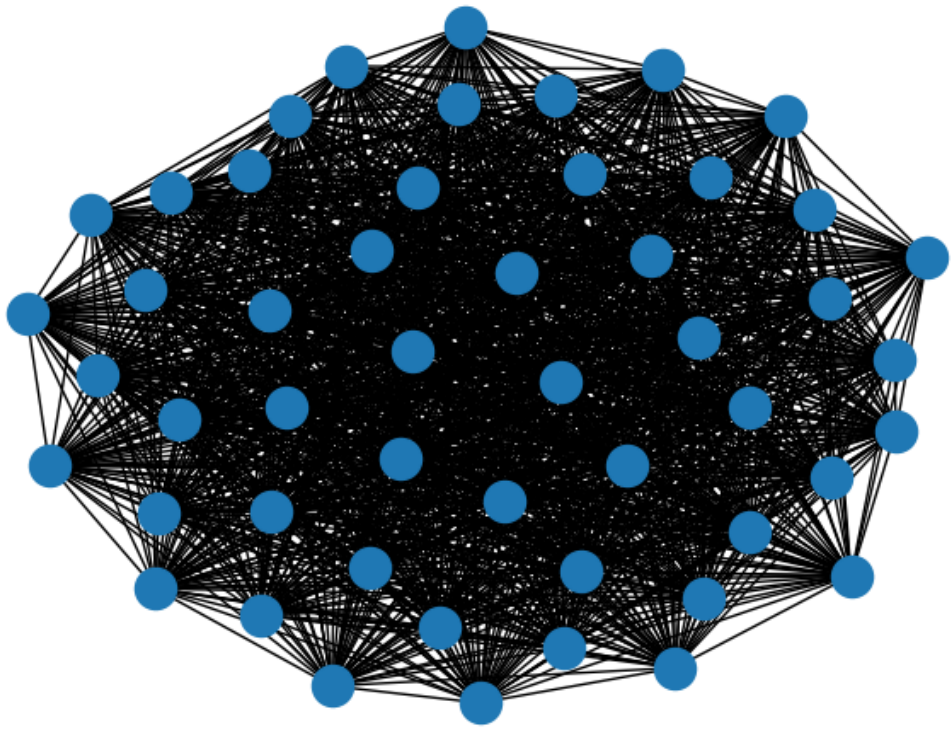


[2]:
```
import networkx as nx
import matplotlib.pyplot as plt
G=nx.Graph()
l=[1,2,3]
G.add_nodes_from(l)
nx.draw(G)
```

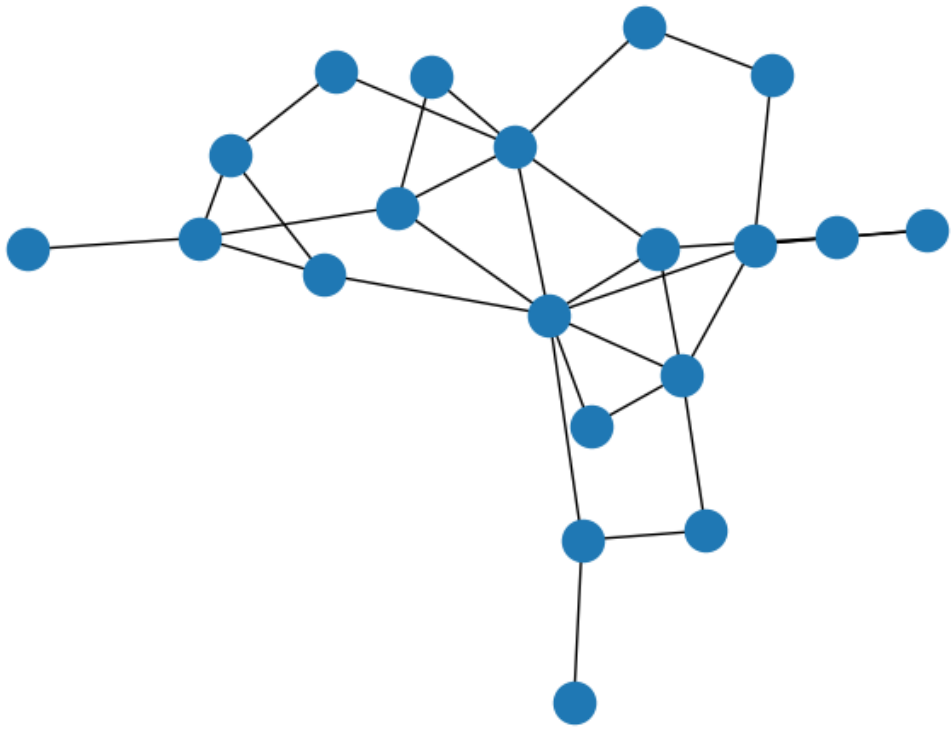
[5]:
```
import networkx as nx
import matplotlib.pyplot as plt
G=nx.complete_graph(10)
nx.draw(G)
```



[6]:
```
import networkx as nx
import matplotlib.pyplot as plt
G=nx.complete_graph(50)
nx.draw(G)
```



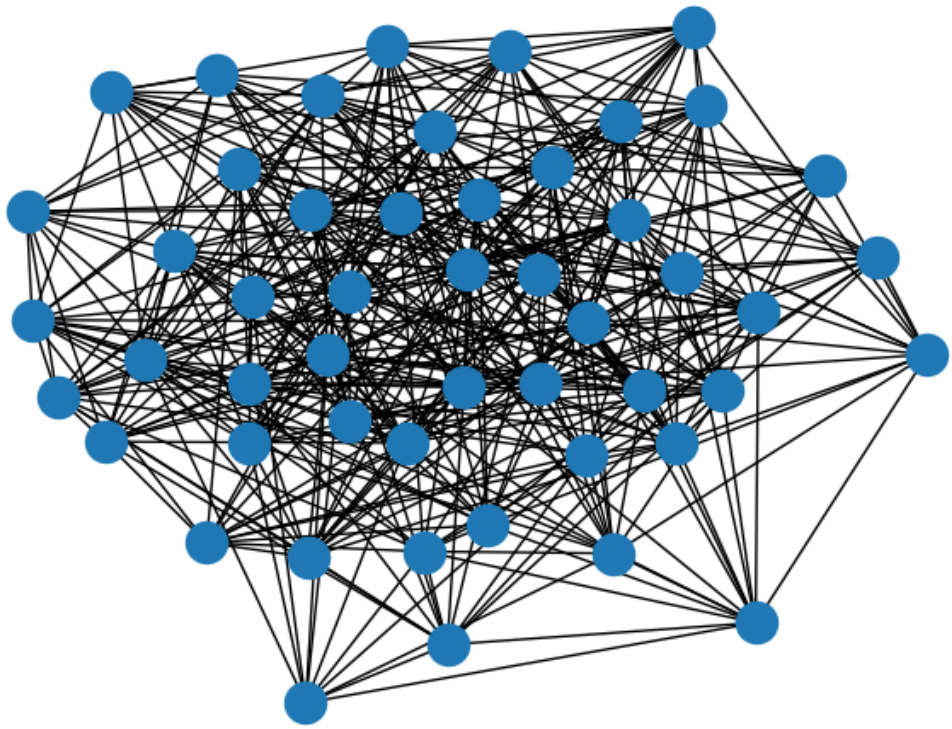
[9]:
```
import networkx as nx
import matplotlib.pyplot as plt
G=nx.gnp_random_graph(20,0.2)
nx.draw(G)
```



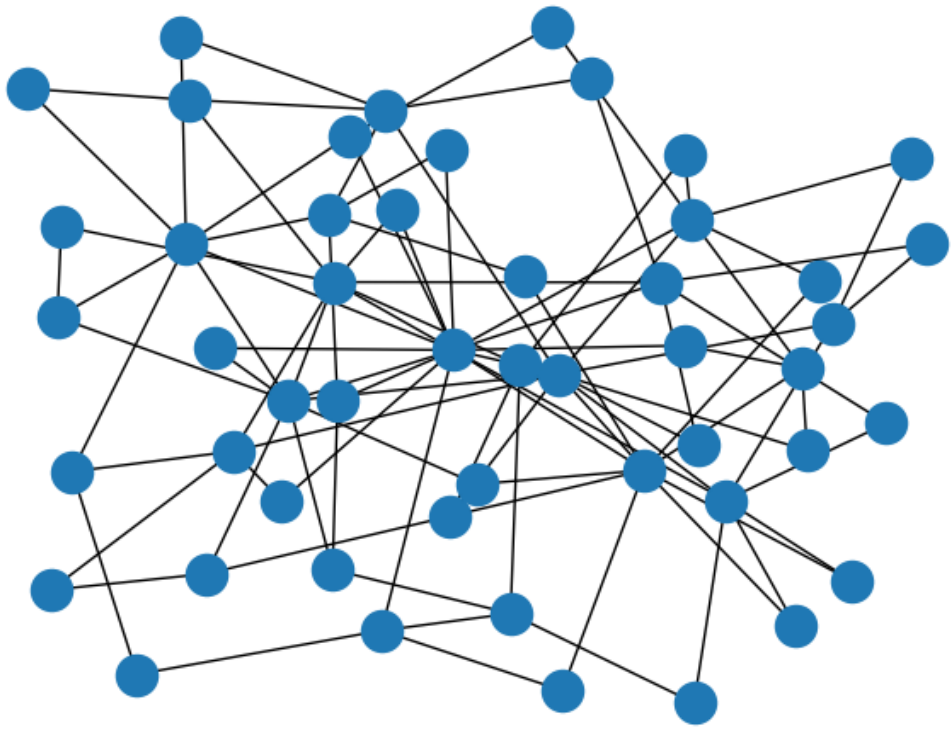
[11]:
```
import networkx as nx
import matplotlib.pyplot as plt
G=nx.gnp_random_graph(50,0.3)
nx.draw(G)
print(G.nodes())
print(G.edges())
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41,
42, 43, 44, 45, 46, 47, 48, 49]
[(0, 1), (0, 3), (0, 4), (0, 10), (0, 11), (0, 13), (0, 21), (0, 24), (0, 26),
(0, 28), (0, 30), (0, 31), (0, 32), (0, 33), (0, 37), (0, 40), (0, 42), (0, 46),
(1, 3), (1, 4), (1, 5), (1, 6), (1, 9), (1, 10), (1, 13), (1, 15), (1, 18), (1,
21), (1, 25), (1, 26), (1, 27), (1, 31), (1, 34), (1, 38), (1, 41), (1, 43), (1,
44), (1, 45), (1, 48), (2, 7), (2, 10), (2, 12), (2, 15), (2, 16), (2, 17), (2,
18), (2, 19), (2, 20), (2, 24), (2, 31), (2, 32), (2, 33), (2, 34), (2, 35), (2,
40), (2, 44), (2, 46), (3, 5), (3, 6), (3, 7), (3, 8), (3, 10), (3, 11), (3,
12), (3, 14), (3, 16), (3, 17), (3, 18), (3, 27), (3, 30), (3, 33), (3, 35), (3,
42), (3, 45), (3, 47), (4, 5), (4, 7), (4, 16), (4, 18), (4, 21), (4, 22), (4,
25), (4, 27), (4, 29), (4, 30), (4, 33), (4, 36), (4, 39), (4, 41), (4, 42), (4,
45), (4, 48), (5, 7), (5, 11), (5, 12), (5, 14), (5, 19), (5, 20), (5, 24), (5,
```

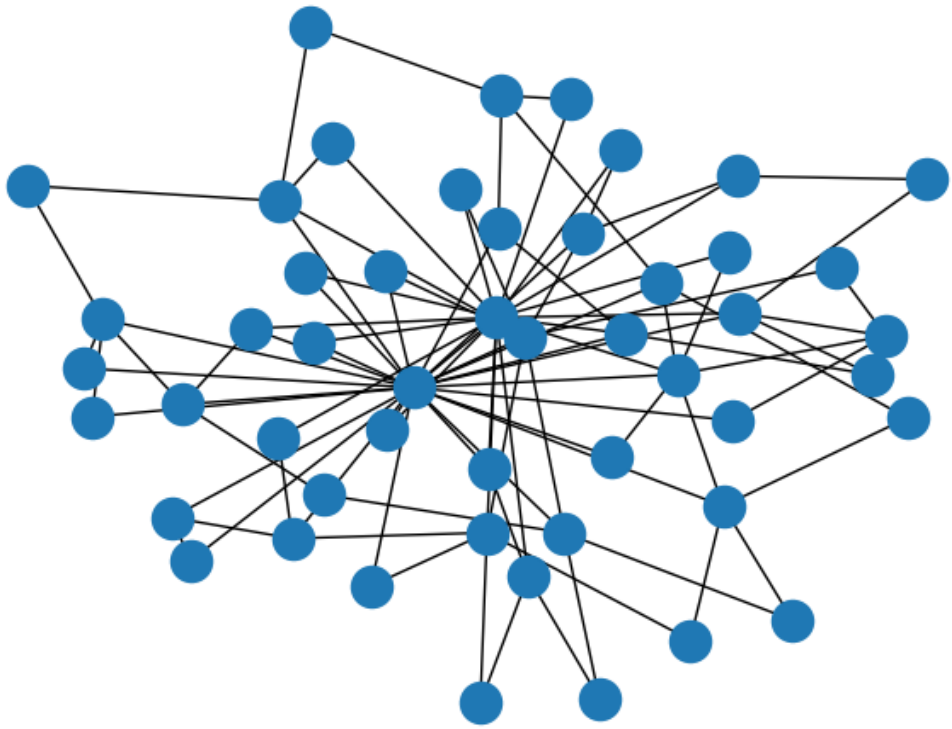
26), (5, 28), (5, 30), (5, 35), (5, 40), (5, 41), (6, 9), (6, 10), (6, 16), (6, 17), (6, 27), (6, 29), (6, 30), (6, 31), (6, 38), (6, 39), (6, 40), (6, 44), (6, 48), (7, 16), (7, 18), (7, 19), (7, 22), (7, 23), (7, 28), (7, 30), (7, 33), (7, 35), (7, 37), (7, 42), (7, 48), (8, 9), (8, 11), (8, 12), (8, 14), (8, 24), (8, 26), (8, 36), (8, 42), (8, 46), (8, 47), (8, 48), (9, 10), (9, 12), (9, 16), (9, 18), (9, 19), (9, 20), (9, 26), (9, 30), (9, 31), (9, 32), (9, 35), (9, 43), (9, 46), (9, 47), (10, 16), (10, 17), (10, 18), (10, 19), (10, 31), (10, 41), (10, 44), (10, 48), (11, 12), (11, 14), (11, 15), (11, 22), (11, 23), (11, 26), (11, 27), (11, 28), (11, 32), (11, 36), (11, 46), (12, 13), (12, 16), (12, 20), (12, 25), (12, 27), (12, 28), (12, 31), (12, 33), (12, 35), (12, 40), (12, 42), (13, 15), (13, 16), (13, 23), (13, 25), (13, 29), (13, 32), (13, 35), (13, 37), (13, 43), (13, 44), (13, 46), (13, 47), (14, 20), (14, 22), (14, 26), (14, 31), (14, 33), (14, 34), (14, 36), (14, 39), (14, 45), (15, 23), (15, 25), (15, 32), (15, 33), (15, 37), (15, 40), (15, 41), (15, 42), (15, 43), (15, 49), (16, 18), (16, 23), (16, 25), (16, 26), (16, 27), (16, 28), (16, 29), (16, 37), (16, 38), (16, 40), (16, 43), (16, 44), (16, 45), (16, 48), (16, 49), (17, 19), (17, 21), (17, 25), (17, 26), (17, 28), (17, 31), (17, 38), (17, 39), (17, 41), (17, 43), (17, 44), (17, 45), (17, 47), (18, 20), (18, 25), (18, 26), (18, 27), (18, 29), (18, 30), (18, 32), (18, 35), (18, 36), (18, 39), (18, 41), (18, 45), (19, 20), (19, 21), (19, 24), (19, 26), (19, 27), (19, 28), (19, 35), (19, 36), (19, 39), (19, 41), (19, 43), (19, 44), (19, 45), (20, 21), (20, 27), (20, 30), (20, 31), (20, 33), (20, 35), (20, 42), (20, 46), (21, 22), (21, 23), (21, 24), (21, 26), (21, 28), (21, 40), (21, 43), (21, 46), (21, 48), (22, 26), (22, 27), (22, 33), (22, 36), (22, 37), (22, 38), (22, 39), (22, 45), (22, 47), (23, 26), (23, 27), (23, 28), (23, 29), (23, 30), (23, 34), (23, 35), (23, 38), (23, 40), (23, 41), (23, 42), (23, 44), (23, 49), (24, 25), (24, 31), (24, 34), (24, 36), (24, 37), (24, 38), (24, 39), (24, 46), (24, 49), (25, 26), (25, 28), (25, 29), (25, 31), (25, 33), (25, 36), (25, 37), (25, 40), (25, 42), (25, 43), (25, 44), (25, 48), (26, 29), (26, 35), (26, 37), (26, 41), (26, 42), (26, 44), (26, 49), (27, 34), (27, 37), (27, 39), (27, 42), (27, 43), (27, 47), (28, 35), (28, 40), (28, 44), (28, 46), (28, 49), (29, 32), (29, 33), (29, 35), (29, 36), (29, 37), (29, 42), (29, 48), (29, 49), (30, 33), (30, 34), (30, 36), (30, 40), (30, 43), (30, 48), (31, 33), (31, 34), (31, 35), (31, 36), (31, 41), (31, 49), (32, 40), (32, 41), (32, 43), (32, 44), (32, 47), (33, 36), (33, 39), (33, 42), (33, 44), (33, 48), (34, 35), (34, 36), (34, 46), (34, 48), (35, 39), (35, 43), (35, 45), (35, 47), (36, 40), (36, 43), (36, 46), (36, 49), (37, 38), (37, 42), (37, 47), (38, 39), (38, 40), (38, 47), (39, 40), (39, 46), (39, 47), (40, 42), (40, 43), (41, 42), (42, 43), (42, 48), (43, 45), (44, 45), (44, 49), (45, 47), (45, 48), (45, 49), (46, 48), (46, 49), (47, 49), (48, 49)]



[1]:
```
import networkx as nx
import matplotlib.pyplot as plt
G=nx.barabasi_albert_graph(50,2)
nx.draw(G)
```



[2]:
```
import networkx as nx
import matplotlib.pyplot as plt
G=nx.barabasi_albert_graph(50,2)
nx.draw(G)
nx.write_gexf(G,"analysis.gexf")
```



4 Six degree of Separation

[ ]:
```
import networkx as nx
import numpy as np
G=nx.read_edgelist("facebook_combined.txt")
N=list(G.nodes())
spll=[]
for u in N:
    for v in N:
        if u!=v:
            l=nx.shortest_path_length(G,u,v)
            #print("Shortest path between ",u," and ",v,"is length :",l)
            spll.append(l)

min_spl=min(spll)
max_spl=max(spll)
avg_spl=np.average(spll)
print("Minimum shortest path length:",min_spl)
print("Maxmum shortest path length:",max_spl)
```

```
print("Average shortest path length:",avg_spl)
```

5 Area Calculation

[1]:
```
import numpy as np
from PIL import Image
width=5
height=4
array=np.zeros([height,width,3],dtype=np.uint8)
img=Image.fromarray(array)
img.save('test.png')
```

[4]:
```
import numpy as np
from PIL import Image
width=5
height=4
array=np.zeros([height,width,3],dtype=np.uint8)
img=Image.fromarray(array)
img.save('test.png')
array1=np.zeros([100,200,3],dtype=np.uint8)
array1[:,:100]=[255,128,0] #orange
array1[:,100]=[0,0,255] #blue
img=Image.fromarray(array1)
img.save('test1.png')
```

[6]:
```
from PIL import Image
im=Image.open('test1.png')
rgb_im=im.convert('RGB')
r,g,b=rgb_im.getpixel((150,1))
print(r,g,b)
```

```
0 0 0
```

[11]:
```
import imageio.v2
from PIL import Image
import numpy as np
import random

img=imageio.v2.imread("map.jpg")

pun=0
ind=0
count=0
while count<10000:
    x=random.randint(0,1023)
    y=random.randint(0,1023)
```

```
    z=0
    if img[x][y][z]==119:
        ind=ind+1
        count=count+1
    else:
        if img[x][y][z]==189:
            pun=pun+1
            count=count+1
area=(pun/ind)*3287263
area
```

[11]: 28181.276348966214

[13]:
```
import imageio.v2
from PIL import Image
import numpy as np
import random

im=imageio.v2.imread("map.jpg")
rgb_im=im.convert("RGB")

count_pun=0
count_in=0
count=0
while count<10000:
    x=random.randint(0,716)
    y=random.randint(0,961)
    z=0
    r,g,b=rgb_im.getpixel((x,y))
    if r==80:
        count_in=count_in+1
        count=count+1
    else:
        if img[x][y][z]==60:
            count_pun=count_pun+1
            count=count+1
area=(count_pun/count_in)*3287263
area
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[13], line 7
      4 import random
      6 im=imageio.v2.imread("map.jpg")
----> 7 rgb_im=im.convert("RGB")
      9 count_pun=0
     10 count_in=0
```

```
AttributeError: 'numpy.ndarray' object has no attribute 'convert'
```

6 FLAMES

[19]:
```
a=input()
b=input()
l=len(a)+len(b)
for i in range(len(a)):
    for j in range(len(b)):
        if a[i]==b[j]:
            l=l-2
print(l)
```

```
 sneha
 sneaa
```

```
0
```

[15]:
```
import string
s="Raj"
print(s.lower())
sl=list(s)
print(sl)
```

```
raj
['R', 'a', 'j']
```

[16]:
```
print(s.replace('j','#'))
```

```
Ra#
```

[17]:
```
l=['a','b','c','d','e','f']
l[1:4]
```

[17]:
```
['b', 'c', 'd']
```

[18]:
```
print(l.index('d'))
```

```
3
```

[6]:
```
import string
def remove_match(l1,l2):
    for i in range(len(l1)):
        for j in range(len(l2)):
            if l1[i]==l2[j]:
```

```
                c=l1[i]
                l1.remove(c)
                l2.remove(c)
                l=l1+['*']+l2
                return [l,True]
    l=l1+['*']+l2
    return [l,False]


p1=input("Enter name1:")
p1=p1.lower()
p1=p1.replace(" ","")

p2=input("Enter name2:")
p2=p2.lower()
p2=p2.replace(" ","")

l1=list(p1)
l2=list(p2)
proceed=True
while proceed:
    final_list=remove_match(l1,l2)
    con_list=final_list[0]
    proceed=final_list[1]
    star_index=con_list.index('*')
    l1=con_list[:star_index]
    l2=con_list[star_index+1:]

count=len(l1)+len(l2)
result=["Friends","Love","Affection","Marriage","Enemy","Siblings"]

while(len(result)>1):
    split_index=(count%len(result))-1
    if split_index>=0:
        right=result[split_index+1:]
        left=result[:split_index]
        result=right+left
    else:
        result=result[:len(result)-1]

print(result[0])
```

```
Enter name1: sneha
Enter name2: sai
```

```
Enemy
```

7 Data Compression

[3]:
```
 import numpy as np
a=np.array([1,2,3])
print(type(a))
print(a.shape)
print(a[0],a[1],a[2])
a[1]=6
print(a)
```

```
<class 'numpy.ndarray'>
(3,)
1 2 3
[1 6 3]
```

[4]:
```
a=np.array([[1,2,3],[4,5,6]])
print(a.shape)
```

```
(2, 3)
```

[5]:
```
a=np.zeros((2,2))
a
```

[5]:
```
array([[0., 0.],
       [0., 0.]])
```

[6]:
```
b=np.ones((2,2))
b
```

[6]:
```
array([[1., 1.],
       [1., 1.]])
```

[7]:
```
c=np.full((2,2),6)
c
```

[7]:
```
array([[6, 6],
       [6, 6]])
```

[8]:
```
d=np.random.random((2,2))
d
```

[8]:
```
array([[0.10822671, 0.87571574],
       [0.87199719, 0.69663803]])
```

[15]:
```
import numpy as np
x=np.array([1,2])
print(x.dtype)
```

```
int64
```

[16]:
```
x=np.array([1,2],dtype=np.int64)
print(x.dtype)
```

```
int64
```

[19]:
```
import numpy as np
x=np.array([[1,2],[3,4]],dtype=np.float64)
y=np.array([[5,6],[7,8]],dtype=np.float64)
print(x)
y
```

```
[[1. 2.]
 [3. 4.]]
```

[19]:
```
array([[5., 6.],
       [7., 8.]])
```

[21]:
```
print(x+y)
np.add(x,y)
```

```
[[ 6.  8.]
 [10. 12.]]
```

[21]:
```
array([[ 6.,  8.],
       [10., 12.]])
```

[22]:
```
print(x-y)
np.subtract(x,y)
```

```
[[-4. -4.]
 [-4. -4.]]
```

[22]:
```
array([[-4., -4.],
       [-4., -4.]])
```

[23]:
```
print(x*y)
np.multiply(x,y)
```

```
[[ 5. 12.]
 [21. 32.]]
```

[23]:
```
array([[ 5., 12.],
       [21., 32.]])
```

[24]:
```
x/y
```

[24]:
```
array([[0.2       , 0.33333333],
       [0.42857143, 0.5       ]])
```

[25]:
```
np.sqrt(x)
```

[25]:
```
array([[1.        , 1.41421356],
       [1.73205081, 2.        ]])
```

[26]:
```
x.T
```

[26]:
```
array([[1., 3.],
       [2., 4.]])
```

[27]:
```
np.sum(x)
```

[27]:
```
10.0
```

[29]:
```
print(np.sum(x,axis=0))
np.sum(x,axis=1)
```

```
[4. 6.]
```

[29]:
```
array([3., 7.])
```

[40]:
```
import numpy as np
from PIL import Image
im=Image.open("enhanced.png")
pixelMap=im.load()
I=np.asanyarray(Image.open("enhanced.png"))
I
```

[40]:
```
array([[  5,   5,   5, …,   7,   2,   0],
       [  5,   5,   5, …,   7,   2,   0],
       [  5,   5,   5, …,   7,   2,   0],
       …,
       [255, 255, 254, …, 255, 254, 254],
       [255, 255, 255, …, 255, 254, 254],
       [255, 255, 255, …, 255, 254, 254]], dtype=uint8)
```

[41]:
```
import numpy as np
from PIL import Image
im=Image.open("enhanced.png")
pixelMap=im.load()
img=Image.new(im.mode,im.size)
pixelNew=img.load()
# im.show()
# img.show()
```

[43]:
```
for i in range(img.size[0]):
    for j in range(img.size[1]):
        if (pixelMap[i,j]>=0 and pixelMap[i,j]<=31):
            pixelNew[i,j]=0
        elif (pixelMap[i,j]>=32 and pixelMap[i,j]<=63):
            pixelNew[i,j]=1
        elif (pixelMap[i,j]>=64 and pixelMap[i,j]<=95):
            pixelNew[i,j]=2
        elif (pixelMap[i,j]>=96 and pixelMap[i,j]<=127):
            pixelNew[i,j]=3
        elif (pixelMap[i,j]>=128 and pixelMap[i,j]<=159):
            pixelNew[i,j]=4
        elif (pixelMap[i,j]>=160 and pixelMap[i,j]<=191):
            pixelNew[i,j]=5
        elif (pixelMap[i,j]>=192 and pixelMap[i,j]<=223):
            pixelNew[i,j]=6
        elif (pixelMap[i,j]>=224 and pixelMap[i,j]<=255):
            pixelNew[i,j]=7

img.save("enhanced_1.png")
J=np.asanyarray(Image.open("enhanced_1.png"))
J
```

[43]:
```
array([[0, 0, 0, …, 0, 0, 0],
       [0, 0, 0, …, 0, 0, 0],
       [0, 0, 0, …, 0, 0, 0],
       …,
       [7, 7, 7, …, 7, 7, 7],
       [7, 7, 7, …, 7, 7, 7],
       [7, 7, 7, …, 7, 7, 7]], dtype=uint8)
```

8 Whatsapp using Python-Browser automation

[1]:
```
from selenium import webdriver
```

[6]:
```
a=webdriver.Chrome("chromedriver")
a.get("https://www.seleniumhq.org")
elem=a.find_element_by_link_text("Download")
elem.click()
search=a.find_element_by_id('q')
search.send_keys("Download")
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[6], line 1
----> 1 a=webdriver.Chrome("chromedriver")
```

```
      2 a.get("https://www.seleniumhq.org")
      3 elem=a.find_element_by_link_text("Download")

File ~/.local/lib/python3.10/site-packages/selenium/webdriver/chrome/webdriver.
 ↪py:45, in WebDriver.__init__(self, options, service, keep_alive)
     42 service = service if service else Service()
     43 options = options if options else Options()
---> 45 super().__init__(
     46     browser_name=DesiredCapabilities.CHROME["browserName"],
     47     vendor_prefix="goog",
     48     options=options,
     49     service=service,
     50     keep_alive=keep_alive,
     51 )

File ~/.local/lib/python3.10/site-packages/selenium/webdriver/chromium/webdrive
 ↪py:50, in ChromiumDriver.__init__(self, browser_name, vendor_prefix, options,
 ↪service, keep_alive)
     47 self.service = service
     49 finder = DriverFinder(self.service, options)
---> 50 if finder.get_browser_path():
     51     options.binary_location = finder.get_browser_path()
     52     options.browser_version = None

File ~/.local/lib/python3.10/site-packages/selenium/webdriver/common/
 ↪driver_finder.py:47, in DriverFinder.get_browser_path(self)
     46 def get_browser_path(self) -> str:
---> 47     return self._binary_paths()["browser_path"]

File ~/.local/lib/python3.10/site-packages/selenium/webdriver/common/
 ↪driver_finder.py:56, in DriverFinder._binary_paths(self)
     53 if self._paths["driver_path"]:
     54     return self._paths
---> 56 browser = self._options.capabilities["browserName"]
     57 try:
     58     path = self._service.path

AttributeError: 'str' object has no attribute 'capabilities'
```

[11]:
```
from selenium import webdriver
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as ex
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.by import By
import time
driver=webdriver("/usr/lib/chromedriver_linux64/chromedriver")
driver.get("https://whatsapp.com/")
```

```
wait=WebDriverWait(driver,600)
target='"Anna"'
string="Message using python"
x_arg = '//span[contains(@title, ' + target + ')]'
target=wait.until(ec.presence_of_element_located((By.XPATH,x_arg)))
target.click()

input_box=driver.find_element_by_class_name('_1Plpp')
for i in range(10):
    input_box.send_keys(string+Keys.ENTER)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[11], line 7
      5 from selenium.webdriver.common.by import By
      6 import time
----> 7 driver=webdriver("/usr/lib/chromedriver_linux64/chromedriver")
      8 driver.get("https://whatsapp.com/")
      9 wait=WebDriverWait(driver,600)

TypeError: 'module' object is not callable
```

[1]:
```
from datetime import datetime as dt
import datetime
dt.now()
```

[1]:
```
datetime.datetime(2024, 10, 31, 14, 47, 12, 775715)
```

[2]:
```
print(dt.now())
```

```
2024-10-31 14:47:41.505969
```

[4]:
```
import pytz
tz=pytz.timezone("Singapore")
print(dt.now(tz))
```

```
2024-10-31 17:18:50.029770+08:00
```

[5]:
```
pytz.all_timezones
```

[5]:
```
['Africa/Abidjan',
 'Africa/Accra',
 'Africa/Addis_Ababa',
 'Africa/Algiers',
 'Africa/Asmara',
 'Africa/Asmera',
 'Africa/Bamako',
```

```
'Africa/Bangui',
'Africa/Banjul',
'Africa/Bissau',
'Africa/Blantyre',
'Africa/Brazzaville',
'Africa/Bujumbura',
'Africa/Cairo',
'Africa/Casablanca',
'Africa/Ceuta',
'Africa/Conakry',
'Africa/Dakar',
'Africa/Dar_es_Salaam',
'Africa/Djibouti',
'Africa/Douala',
'Africa/El_Aaiun',
'Africa/Freetown',
'Africa/Gaborone',
'Africa/Harare',
'Africa/Johannesburg',
'Africa/Juba',
'Africa/Kampala',
'Africa/Khartoum',
'Africa/Kigali',
'Africa/Kinshasa',
'Africa/Lagos',
'Africa/Libreville',
'Africa/Lome',
'Africa/Luanda',
'Africa/Lubumbashi',
'Africa/Lusaka',
'Africa/Malabo',
'Africa/Maputo',
'Africa/Maseru',
'Africa/Mbabane',
'Africa/Mogadishu',
'Africa/Monrovia',
'Africa/Nairobi',
'Africa/Ndjamena',
'Africa/Niamey',
'Africa/Nouakchott',
'Africa/Ouagadougou',
'Africa/Porto-Novo',
'Africa/Sao_Tome',
'Africa/Timbuktu',
'Africa/Tripoli',
'Africa/Tunis',
'Africa/Windhoek',
```

```
'America/Adak',
'America/Anchorage',
'America/Anguilla',
'America/Antigua',
'America/Araguaina',
'America/Argentina/Buenos_Aires',
'America/Argentina/Catamarca',
'America/Argentina/ComodRivadavia',
'America/Argentina/Cordoba',
'America/Argentina/Jujuy',
'America/Argentina/La_Rioja',
'America/Argentina/Mendoza',
'America/Argentina/Rio_Gallegos',
'America/Argentina/Salta',
'America/Argentina/San_Juan',
'America/Argentina/San_Luis',
'America/Argentina/Tucuman',
'America/Argentina/Ushuaia',
'America/Aruba',
'America/Asuncion',
'America/Atikokan',
'America/Atka',
'America/Bahia',
'America/Bahia_Banderas',
'America/Barbados',
'America/Belem',
'America/Belize',
'America/Blanc-Sablon',
'America/Boa_Vista',
'America/Bogota',
'America/Boise',
'America/Buenos_Aires',
'America/Cambridge_Bay',
'America/Campo_Grande',
'America/Cancun',
'America/Caracas',
'America/Catamarca',
'America/Cayenne',
'America/Cayman',
'America/Chicago',
'America/Chihuahua',
'America/Ciudad_Juarez',
'America/Coral_Harbour',
'America/Cordoba',
'America/Costa_Rica',
'America/Creston',
'America/Cuiaba',
```

```
'America/Curacao',
'America/Danmarkshavn',
'America/Dawson',
'America/Dawson_Creek',
'America/Denver',
'America/Detroit',
'America/Dominica',
'America/Edmonton',
'America/Eirunepe',
'America/El_Salvador',
'America/Ensenada',
'America/Fort_Nelson',
'America/Fort_Wayne',
'America/Fortaleza',
'America/Glace_Bay',
'America/Godthab',
'America/Goose_Bay',
'America/Grand_Turk',
'America/Grenada',
'America/Guadeloupe',
'America/Guatemala',
'America/Guayaquil',
'America/Guyana',
'America/Halifax',
'America/Havana',
'America/Hermosillo',
'America/Indiana/Indianapolis',
'America/Indiana/Knox',
'America/Indiana/Marengo',
'America/Indiana/Petersburg',
'America/Indiana/Tell_City',
'America/Indiana/Vevay',
'America/Indiana/Vincennes',
'America/Indiana/Winamac',
'America/Indianapolis',
'America/Inuvik',
'America/Iqaluit',
'America/Jamaica',
'America/Jujuy',
'America/Juneau',
'America/Kentucky/Louisville',
'America/Kentucky/Monticello',
'America/Knox_IN',
'America/Kralendijk',
'America/La_Paz',
'America/Lima',
'America/Los_Angeles',
```

```
'America/Louisville',
'America/Lower_Princes',
'America/Maceio',
'America/Managua',
'America/Manaus',
'America/Marigot',
'America/Martinique',
'America/Matamoros',
'America/Mazatlan',
'America/Mendoza',
'America/Menominee',
'America/Merida',
'America/Metlakatla',
'America/Mexico_City',
'America/Miquelon',
'America/Moncton',
'America/Monterrey',
'America/Montevideo',
'America/Montreal',
'America/Montserrat',
'America/Nassau',
'America/New_York',
'America/Nipigon',
'America/Nome',
'America/Noronha',
'America/North_Dakota/Beulah',
'America/North_Dakota/Center',
'America/North_Dakota/New_Salem',
'America/Nuuk',
'America/Ojinaga',
'America/Panama',
'America/Pangnirtung',
'America/Paramaribo',
'America/Phoenix',
'America/Port-au-Prince',
'America/Port_of_Spain',
'America/Porto_Acre',
'America/Porto_Velho',
'America/Puerto_Rico',
'America/Punta_Arenas',
'America/Rainy_River',
'America/Rankin_Inlet',
'America/Recife',
'America/Regina',
'America/Resolute',
'America/Rio_Branco',
'America/Rosario',
```

```
'America/Santa_Isabel',
'America/Santarem',
'America/Santiago',
'America/Santo_Domingo',
'America/Sao_Paulo',
'America/Scoresbysund',
'America/Shiprock',
'America/Sitka',
'America/St_Barthelemy',
'America/St_Johns',
'America/St_Kitts',
'America/St_Lucia',
'America/St_Thomas',
'America/St_Vincent',
'America/Swift_Current',
'America/Tegucigalpa',
'America/Thule',
'America/Thunder_Bay',
'America/Tijuana',
'America/Toronto',
'America/Tortola',
'America/Vancouver',
'America/Virgin',
'America/Whitehorse',
'America/Winnipeg',
'America/Yakutat',
'America/Yellowknife',
'Antarctica/Casey',
'Antarctica/Davis',
'Antarctica/DumontDUrville',
'Antarctica/Macquarie',
'Antarctica/Mawson',
'Antarctica/McMurdo',
'Antarctica/Palmer',
'Antarctica/Rothera',
'Antarctica/South_Pole',
'Antarctica/Syowa',
'Antarctica/Troll',
'Antarctica/Vostok',
'Arctic/Longyearbyen',
'Asia/Aden',
'Asia/Almaty',
'Asia/Amman',
'Asia/Anadyr',
'Asia/Aqtau',
'Asia/Aqtobe',
'Asia/Ashgabat',
```

```
'Asia/Ashkhabad',
'Asia/Atyrau',
'Asia/Baghdad',
'Asia/Bahrain',
'Asia/Baku',
'Asia/Bangkok',
'Asia/Barnaul',
'Asia/Beirut',
'Asia/Bishkek',
'Asia/Brunei',
'Asia/Calcutta',
'Asia/Chita',
'Asia/Choibalsan',
'Asia/Chongqing',
'Asia/Chungking',
'Asia/Colombo',
'Asia/Dacca',
'Asia/Damascus',
'Asia/Dhaka',
'Asia/Dili',
'Asia/Dubai',
'Asia/Dushanbe',
'Asia/Famagusta',
'Asia/Gaza',
'Asia/Harbin',
'Asia/Hebron',
'Asia/Ho_Chi_Minh',
'Asia/Hong_Kong',
'Asia/Hovd',
'Asia/Irkutsk',
'Asia/Istanbul',
'Asia/Jakarta',
'Asia/Jayapura',
'Asia/Jerusalem',
'Asia/Kabul',
'Asia/Kamchatka',
'Asia/Karachi',
'Asia/Kashgar',
'Asia/Kathmandu',
'Asia/Katmandu',
'Asia/Khandyga',
'Asia/Kolkata',
'Asia/Krasnoyarsk',
'Asia/Kuala_Lumpur',
'Asia/Kuching',
'Asia/Kuwait',
'Asia/Macao',
```

```
'Asia/Macau',
'Asia/Magadan',
'Asia/Makassar',
'Asia/Manila',
'Asia/Muscat',
'Asia/Nicosia',
'Asia/Novokuznetsk',
'Asia/Novosibirsk',
'Asia/Omsk',
'Asia/Oral',
'Asia/Phnom_Penh',
'Asia/Pontianak',
'Asia/Pyongyang',
'Asia/Qatar',
'Asia/Qostanay',
'Asia/Qyzylorda',
'Asia/Rangoon',
'Asia/Riyadh',
'Asia/Saigon',
'Asia/Sakhalin',
'Asia/Samarkand',
'Asia/Seoul',
'Asia/Shanghai',
'Asia/Singapore',
'Asia/Srednekolymsk',
'Asia/Taipei',
'Asia/Tashkent',
'Asia/Tbilisi',
'Asia/Tehran',
'Asia/Tel_Aviv',
'Asia/Thimbu',
'Asia/Thimphu',
'Asia/Tokyo',
'Asia/Tomsk',
'Asia/Ujung_Pandang',
'Asia/Ulaanbaatar',
'Asia/Ulan_Bator',
'Asia/Urumqi',
'Asia/Ust-Nera',
'Asia/Vientiane',
'Asia/Vladivostok',
'Asia/Yakutsk',
'Asia/Yangon',
'Asia/Yekaterinburg',
'Asia/Yerevan',
'Atlantic/Azores',
'Atlantic/Bermuda',
```

```
'Atlantic/Canary',
'Atlantic/Cape_Verde',
'Atlantic/Faeroe',
'Atlantic/Faroe',
'Atlantic/Jan_Mayen',
'Atlantic/Madeira',
'Atlantic/Reykjavik',
'Atlantic/South_Georgia',
'Atlantic/St_Helena',
'Atlantic/Stanley',
'Australia/ACT',
'Australia/Adelaide',
'Australia/Brisbane',
'Australia/Broken_Hill',
'Australia/Canberra',
'Australia/Currie',
'Australia/Darwin',
'Australia/Eucla',
'Australia/Hobart',
'Australia/LHI',
'Australia/Lindeman',
'Australia/Lord_Howe',
'Australia/Melbourne',
'Australia/NSW',
'Australia/North',
'Australia/Perth',
'Australia/Queensland',
'Australia/South',
'Australia/Sydney',
'Australia/Tasmania',
'Australia/Victoria',
'Australia/West',
'Australia/Yancowinna',
'Brazil/Acre',
'Brazil/DeNoronha',
'Brazil/East',
'Brazil/West',
'CET',
'CST6CDT',
'Canada/Atlantic',
'Canada/Central',
'Canada/Eastern',
'Canada/Mountain',
'Canada/Newfoundland',
'Canada/Pacific',
'Canada/Saskatchewan',
'Canada/Yukon',
```

```
'Chile/Continental',
'Chile/EasterIsland',
'Cuba',
'EET',
'EST',
'EST5EDT',
'Egypt',
'Eire',
'Etc/GMT',
'Etc/GMT+0',
'Etc/GMT+1',
'Etc/GMT+10',
'Etc/GMT+11',
'Etc/GMT+12',
'Etc/GMT+2',
'Etc/GMT+3',
'Etc/GMT+4',
'Etc/GMT+5',
'Etc/GMT+6',
'Etc/GMT+7',
'Etc/GMT+8',
'Etc/GMT+9',
'Etc/GMT-0',
'Etc/GMT-1',
'Etc/GMT-10',
'Etc/GMT-11',
'Etc/GMT-12',
'Etc/GMT-13',
'Etc/GMT-14',
'Etc/GMT-2',
'Etc/GMT-3',
'Etc/GMT-4',
'Etc/GMT-5',
'Etc/GMT-6',
'Etc/GMT-7',
'Etc/GMT-8',
'Etc/GMT-9',
'Etc/GMT0',
'Etc/Greenwich',
'Etc/UCT',
'Etc/UTC',
'Etc/Universal',
'Etc/Zulu',
'Europe/Amsterdam',
'Europe/Andorra',
'Europe/Astrakhan',
'Europe/Athens',
```

```
'Europe/Belfast',
'Europe/Belgrade',
'Europe/Berlin',
'Europe/Bratislava',
'Europe/Brussels',
'Europe/Bucharest',
'Europe/Budapest',
'Europe/Busingen',
'Europe/Chisinau',
'Europe/Copenhagen',
'Europe/Dublin',
'Europe/Gibraltar',
'Europe/Guernsey',
'Europe/Helsinki',
'Europe/Isle_of_Man',
'Europe/Istanbul',
'Europe/Jersey',
'Europe/Kaliningrad',
'Europe/Kiev',
'Europe/Kirov',
'Europe/Kyiv',
'Europe/Lisbon',
'Europe/Ljubljana',
'Europe/London',
'Europe/Luxembourg',
'Europe/Madrid',
'Europe/Malta',
'Europe/Mariehamn',
'Europe/Minsk',
'Europe/Monaco',
'Europe/Moscow',
'Europe/Nicosia',
'Europe/Oslo',
'Europe/Paris',
'Europe/Podgorica',
'Europe/Prague',
'Europe/Riga',
'Europe/Rome',
'Europe/Samara',
'Europe/San_Marino',
'Europe/Sarajevo',
'Europe/Saratov',
'Europe/Simferopol',
'Europe/Skopje',
'Europe/Sofia',
'Europe/Stockholm',
'Europe/Tallinn',
```

```
'Europe/Tirane',
'Europe/Tiraspol',
'Europe/Ulyanovsk',
'Europe/Uzhgorod',
'Europe/Vaduz',
'Europe/Vatican',
'Europe/Vienna',
'Europe/Vilnius',
'Europe/Volgograd',
'Europe/Warsaw',
'Europe/Zagreb',
'Europe/Zaporozhye',
'Europe/Zurich',
'GB',
'GB-Eire',
'GMT',
'GMT+0',
'GMT-0',
'GMT0',
'Greenwich',
'HST',
'Hongkong',
'Iceland',
'Indian/Antananarivo',
'Indian/Chagos',
'Indian/Christmas',
'Indian/Cocos',
'Indian/Comoro',
'Indian/Kerguelen',
'Indian/Mahe',
'Indian/Maldives',
'Indian/Mauritius',
'Indian/Mayotte',
'Indian/Reunion',
'Iran',
'Israel',
'Jamaica',
'Japan',
'Kwajalein',
'Libya',
'MET',
'MST',
'MST7MDT',
'Mexico/BajaNorte',
'Mexico/BajaSur',
'Mexico/General',
'NZ',
```

```
'NZ-CHAT',
'Navajo',
'PRC',
'PST8PDT',
'Pacific/Apia',
'Pacific/Auckland',
'Pacific/Bougainville',
'Pacific/Chatham',
'Pacific/Chuuk',
'Pacific/Easter',
'Pacific/Efate',
'Pacific/Enderbury',
'Pacific/Fakaofo',
'Pacific/Fiji',
'Pacific/Funafuti',
'Pacific/Galapagos',
'Pacific/Gambier',
'Pacific/Guadalcanal',
'Pacific/Guam',
'Pacific/Honolulu',
'Pacific/Johnston',
'Pacific/Kanton',
'Pacific/Kiritimati',
'Pacific/Kosrae',
'Pacific/Kwajalein',
'Pacific/Majuro',
'Pacific/Marquesas',
'Pacific/Midway',
'Pacific/Nauru',
'Pacific/Niue',
'Pacific/Norfolk',
'Pacific/Noumea',
'Pacific/Pago_Pago',
'Pacific/Palau',
'Pacific/Pitcairn',
'Pacific/Pohnpei',
'Pacific/Ponape',
'Pacific/Port_Moresby',
'Pacific/Rarotonga',
'Pacific/Saipan',
'Pacific/Samoa',
'Pacific/Tahiti',
'Pacific/Tarawa',
'Pacific/Tongatapu',
'Pacific/Truk',
'Pacific/Wake',
'Pacific/Wallis',
```

```
 'Pacific/Yap',
 'Poland',
 'Portugal',
 'ROC',
 'ROK',
 'Singapore',
 'Turkey',
 'UCT',
 'US/Alaska',
 'US/Aleutian',
 'US/Arizona',
 'US/Central',
 'US/East-Indiana',
 'US/Eastern',
 'US/Hawaii',
 'US/Indiana-Starke',
 'US/Michigan',
 'US/Mountain',
 'US/Pacific',
 'US/Samoa',
 'UTC',
 'Universal',
 'W-SU',
 'WET',
 'Zulu']
```

[6]:
```
len(pytz.all_timezones)
```

[6]: 596

[9]:
```
import calendar
calendar.weekday?
```

```
Signature: calendar.weekday(year, month, day)
Docstring: Return weekday (0-6 ~ Mon-Sun) for year, month (1-12), day (1-31).
File:      /usr/lib/python3.10/calendar.py
Type:      function
```

[10]:
```
calendar.weekday(2024,4,6)
```

[10]: 5

[ ]:
```
while(1):
    year=int(input("Enter year(1970-2024):"))
    if year<1970:
        print("Entr year in range")
    else:
```

```
        break
while(1):
    month=int(input("Enter month(1-12):"))
    if month<=12 and month>0:
        break
    else:
        print("Enter a valid month:")
def check_leap(y):
    if y%400==0:
        return True
    else:
        if y%4==0:
            return True
        else:
            return False
leap=check_leap(year-
while(1):
    date=int(input("Enter date:"))
    if check_valid_date(date,month,year,leap):
        break
    else:
        print("Enter a valid date")
```